



SDA WEEK 8

INTERACTION DIAGRAMS

Syed Ahmed Khan
syed.ahmed@nu.edu.pk

LECTURE OUTLINE

1. Introduce the concept of Interaction Diagrams.
2. Differentiate between the two main types: Sequence and Collaboration Diagrams.
3. Dive deep into Sequence Diagrams.
4. Learn all the essential notations.
5. Work through examples to solidify these concepts.

INTERACTION DIAGRAM

- They are a type of behavior diagram that describe the dynamic behavior of a system. Think of it as a script or a conversation between a set of objects working together to accomplish a specific purpose.

WHY ARE THEY IMPORTANT?

- Model interactions between objects.
- Understand how a use case is actually realized through a sequence of messages.
- Verify that our class diagram can support the required behavior.
- Identify responsibilities and assign them as operations to specific classes.

INTERACTION DIAGRAM...

There are two primary types of Interaction Diagrams in UML:

1. **Collaboration Diagrams:** These emphasize the structural organization of the objects and their links. We will cover these in detail next week.
2. **Sequence Diagrams:** This is our main focus for today. They emphasize the time ordering of messages.

SEQUENCE DIAGRAMS

- A Sequence Diagram shows how objects interact in a specific situation. It provides an approximation of time by reading the diagram from top to bottom. The higher an event is, the earlier it occurs.
- They are excellent for modeling various flows of control: simple sequences, branching with if-else conditions, loops, recursion, and even concurrency.

SEQUENCE DIAGRAMS

A sequence diagram is structured in such a way that it represents a timeline that begins at the top and descends gradually to mark the sequence of interactions. Each object has a column and the messages exchanged between them are represented by arrows.

SEQUENCE DIAGRAMS

- **Focus:** The central focus is on the sequence of messages.
- **Vertical Axis:** Represents time. As you go down the diagram, time progresses.
- **Horizontal Axis:** Represents the objects (or actors) participating in the interaction.
- **Scenario-Driven:** A sequence diagram typically models one specific scenario or a single path through a use case

EXAMPLE

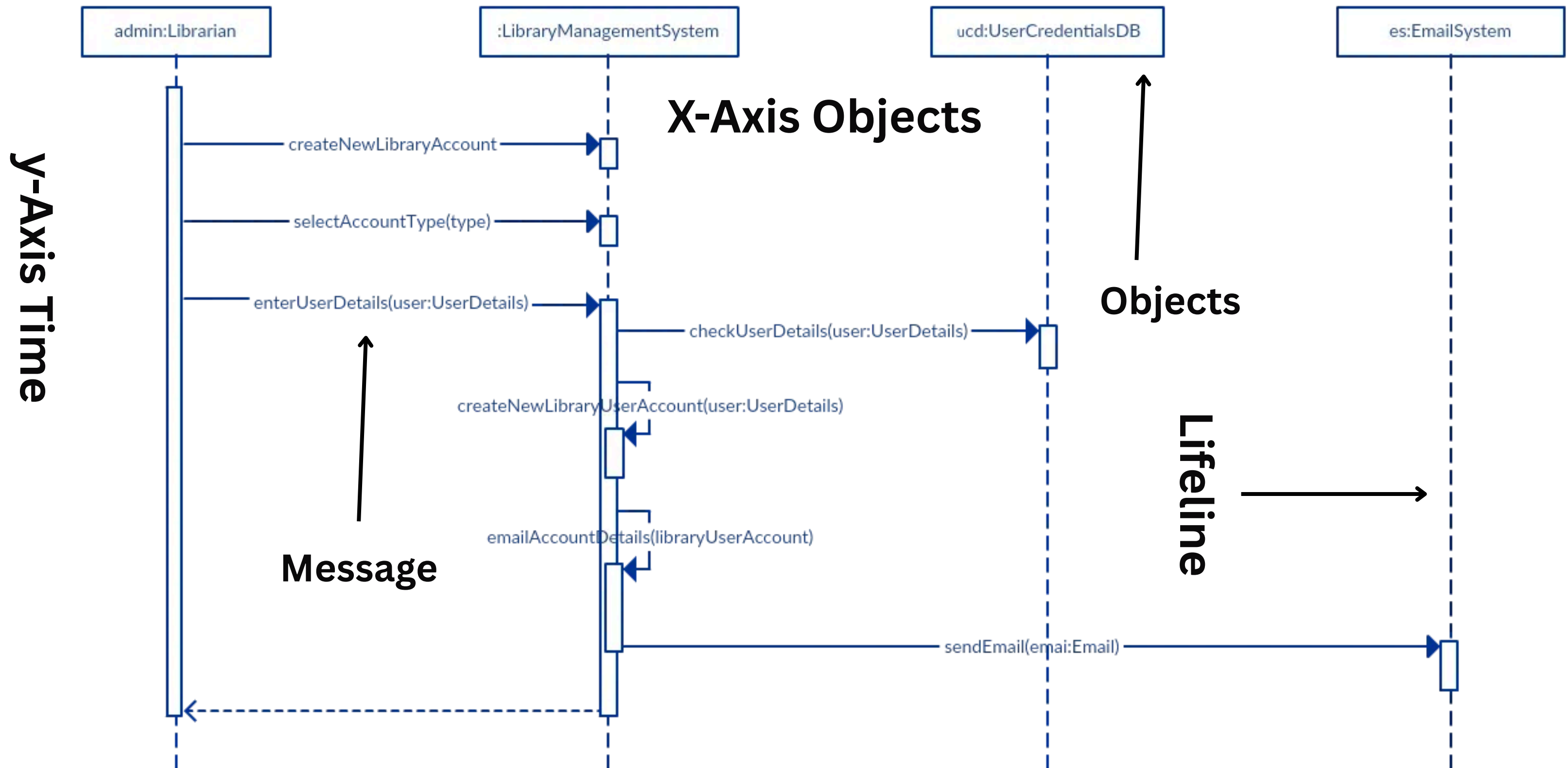
' **Borrow Book**' use case in a library system.

- The Librarian (actor) enters a borrower ID.
- The UI (object) calls a Transaction object to calculate how many books the borrower can have.
- The Transaction object creates a Fine object and calculates any existing fines.
- It also checks the borrower's checked-out and overdue media.
- This information is returned to the UI to be displayed.
- If the borrower's fine amount is zero, the transaction proceeds; otherwise, an invalid message might be shown.

SEQUENCE DIAGRAM NOTATIONS

- Every sequence diagram is enclosed in a frame. In the top-left corner, you see sd followed by the diagram's name in a pentagon. This is the standard UML frame notation for all behavior diagrams.
- Inside the frame, along the top, we place the objects involved. Each object is represented by a rectangle with its name, often underlined, following the format `objectName : ClassName`. You can omit the object name or the class name, but including both is clearest.
- From each object, a dashed vertical line extends downward. This is the object's lifeline. It represents the object's existence over time.
- Communication is shown as horizontal arrows between these lifelines. These are the messages.

SEQUENCE DIAGRAM NOTATIONS



GENERIC AND INSTANCE FORMS

It's important to know that sequence diagrams can be drawn in two forms:

- Instance Form: Shows one specific, concrete execution path. It has no conditions, branches, or loops. For example, 'Joe successfully borrows "Design Patterns"'.
• Generic Form: Shows all possible alternatives for a scenario. It does include conditions, branches, and loops. For example, 'Borrow Book', which includes both successful and failed attempts.

BASIC NOTATIONS: OBJECTS

Objects or Participants :-

The sequence diagram is made up of collection of participants or objects. Participants are system parts that interact each other during sequence diagram.

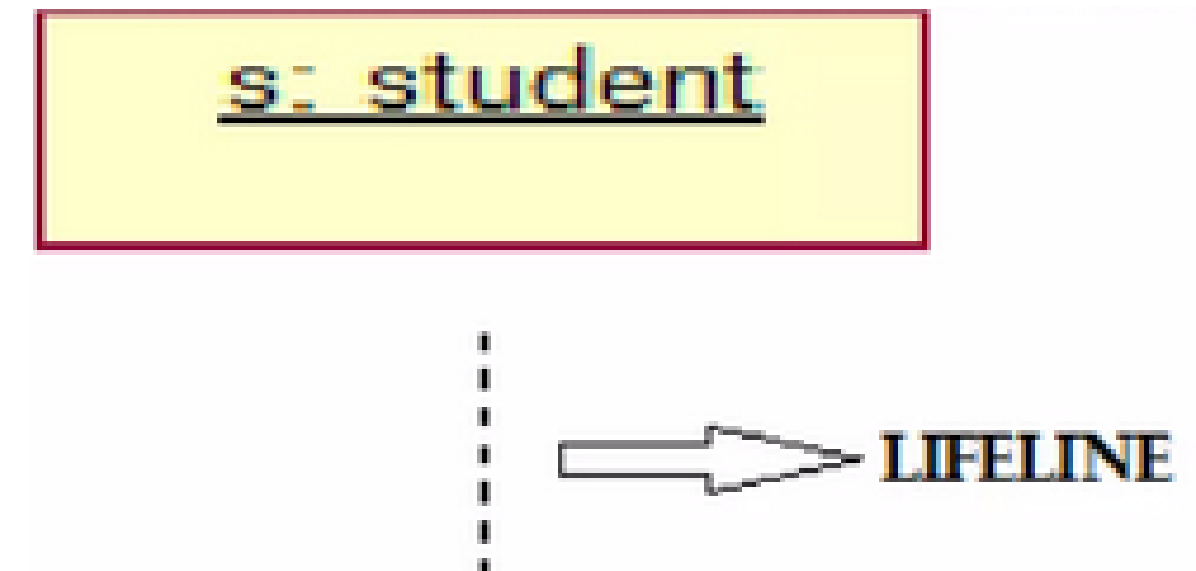
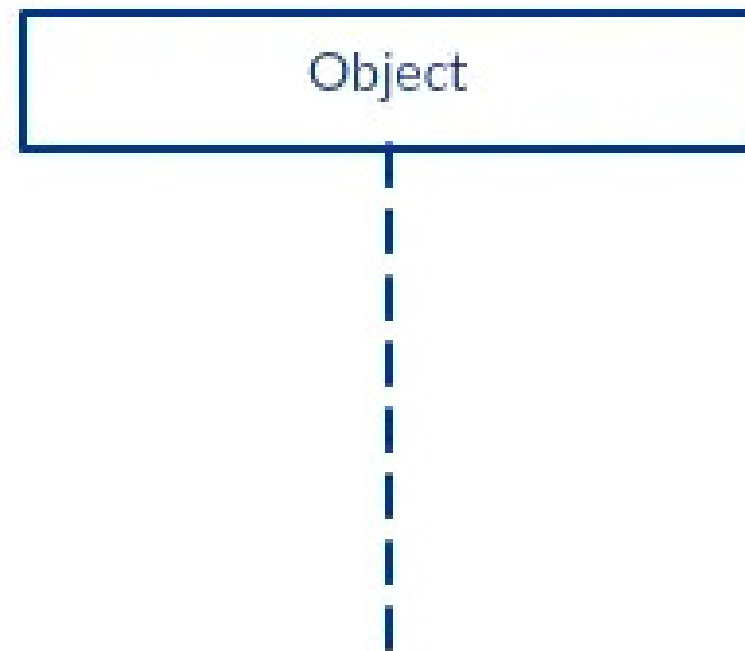
The participants interact with each other by sending and receiving message

The object is represented by as below:

Object:Class_Name

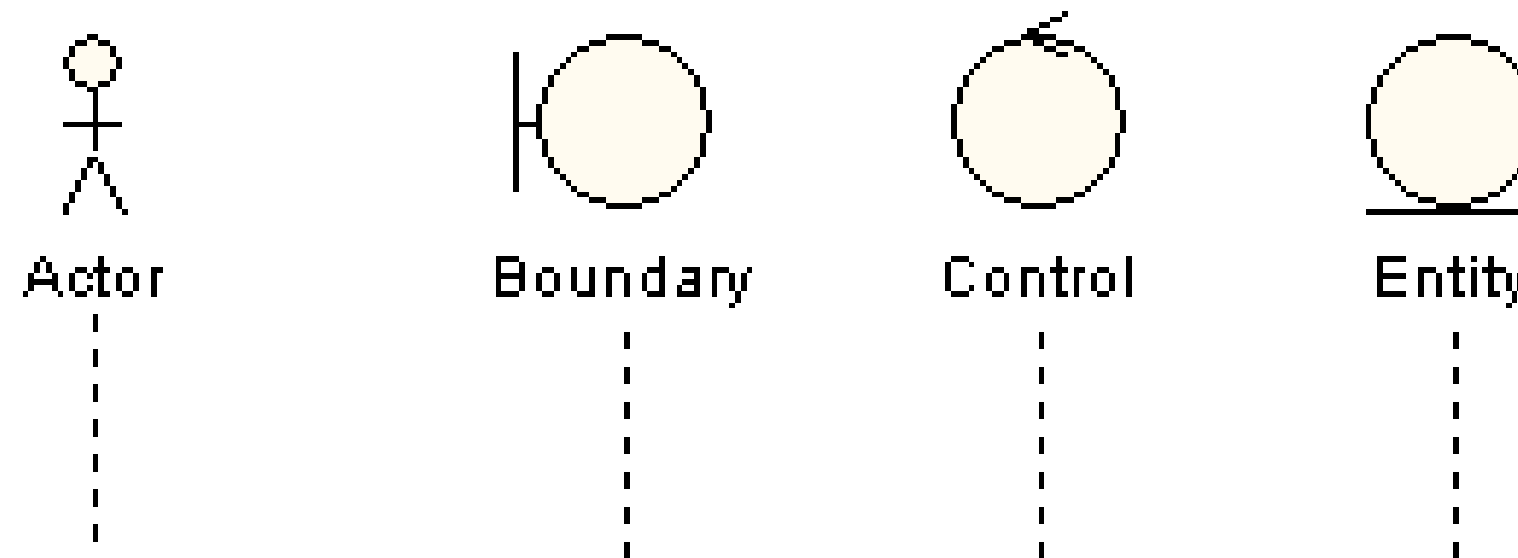
BASIC NOTATIONS: LIFELINE

A sequence diagram is made up of several of these lifeline notations that should be arranged horizontally across the top of the diagram. No two lifeline notations should overlap each other. They represent the different objects or parts that interact with each other in the system during the sequence.



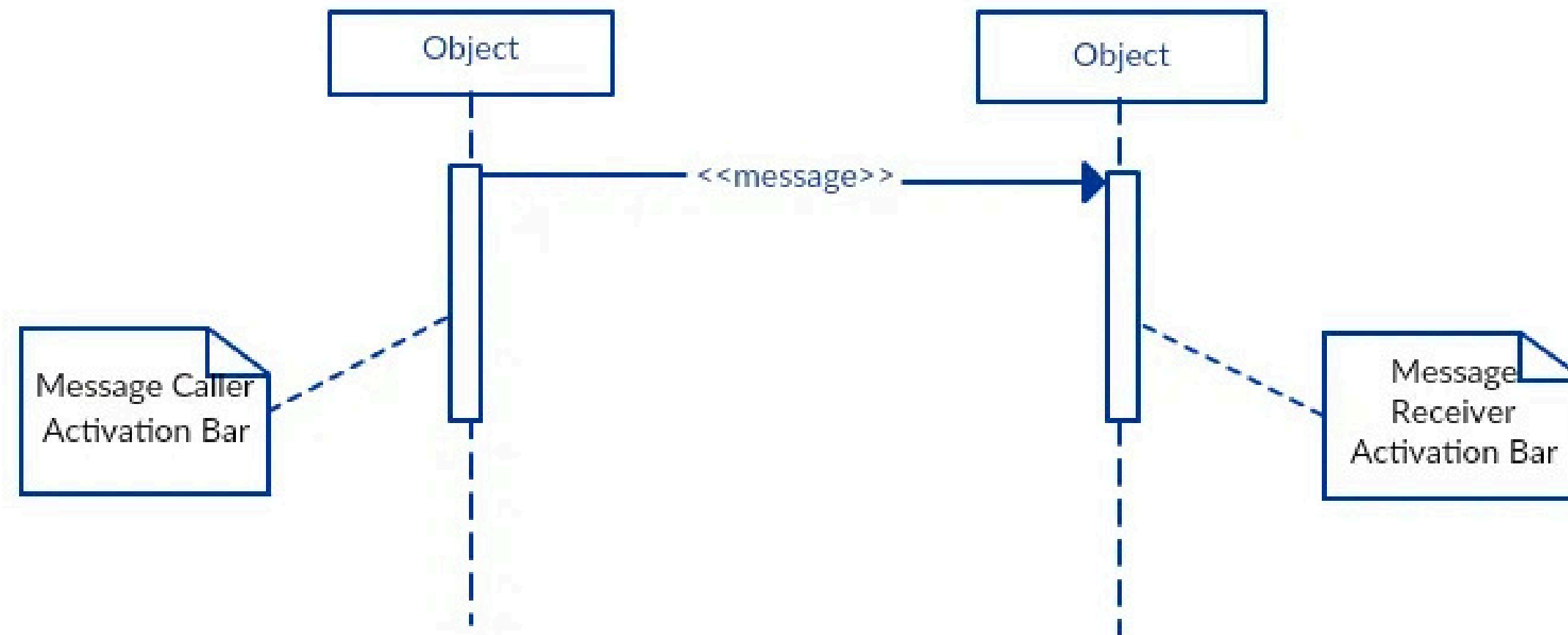
BASIC NOTATIONS: LIFELINE

Sometimes a sequence diagram will have a lifeline with an actor element symbol at its head. This will usually be the case if the sequence diagram is owned by a use case. Boundary, control and entity elements from robustness diagrams can also own lifelines.



BASIC NOTATIONS: ACTIVATION BAR

This thin rectangle on a lifeline shows the period during which the object is performing an operation, either directly or through a subordinate operation. It's a visual cue for 'focus of control'.



BASIC NOTATIONS: MESSAGE

A message is a communication between objects that expects an action. The arrow shows the direction. The processing time is implied by the activation bar.

An arrow from the Message Caller to the Message Receiver specifies a message in a sequence diagram. A message can flow in any direction; from left to right, right to left, or back to the Message Caller itself. While you can describe the message being sent from one object to the other on the arrow, with different arrowheads you can indicate the type of message being sent or received.



BASIC NOTATIONS: MESSAGE

The message arrow comes with a description, which is known as a message signature, on it. The format for this message signature is below. All parts except the message_name are optional.

attribute = message_name (arguments): return_type

TYPES OF MESSAGES

- **Synchronous Message:** A solid line with a solid arrowhead. This is a regular function call. The sender waits for the receiver to finish processing the message and return a response before it continues. This is the most common type.



- **Asynchronous Message:** A solid line with a line arrowhead. The sender does not wait; it fires the message and continues executing immediately. Think of sending a signal or starting a new thread.

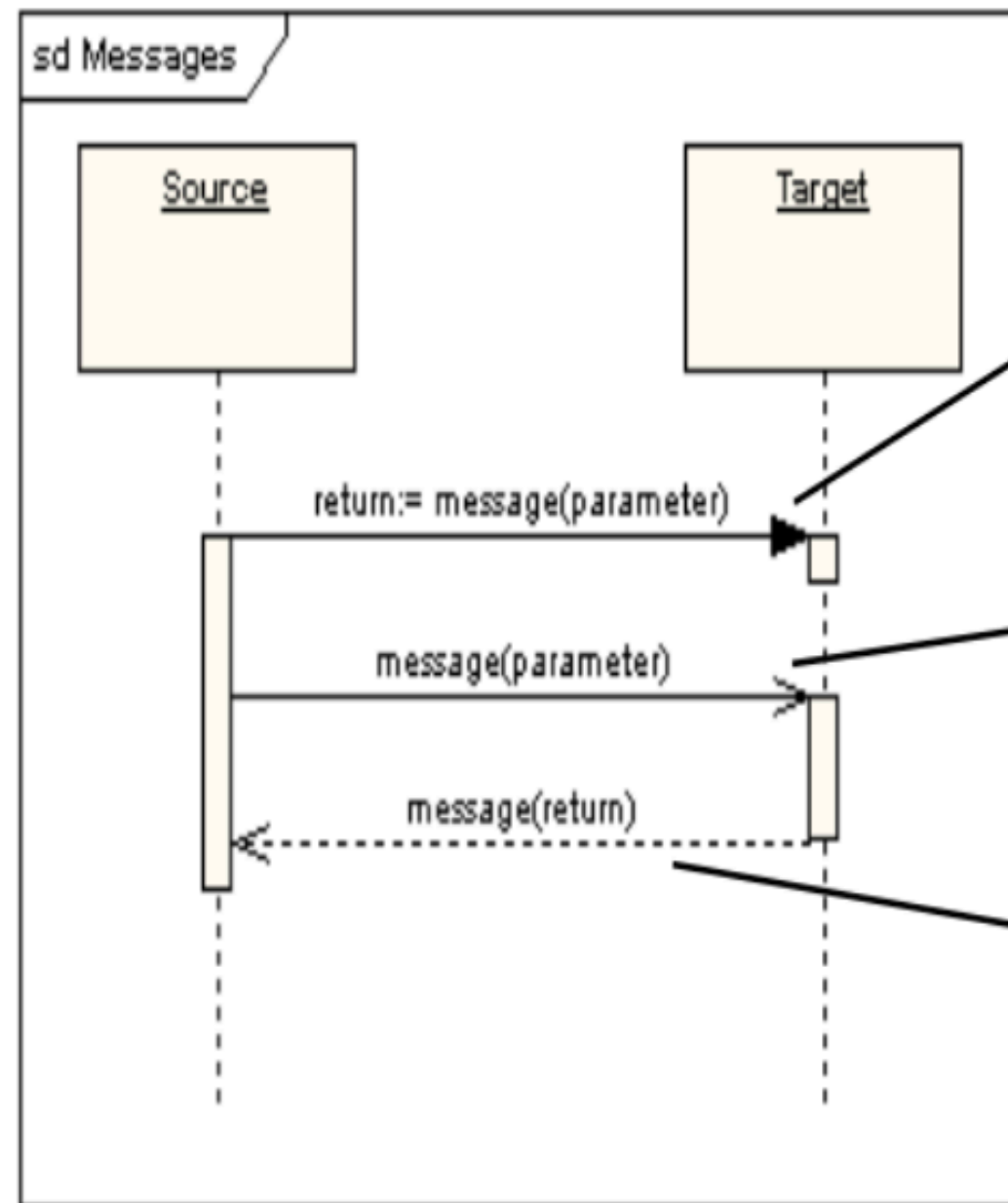


TYPES OF MESSAGES

Reply/Return Message: A dashed line with an open arrowhead. This shows the return from a synchronous message. You can show the return value either on this dashed arrow or as a return variable on the original call (returnVar=message(parameter))



TYPES OF MESSAGES: EXAMPLE



synchronous message line
denoted by the solid arrowhead

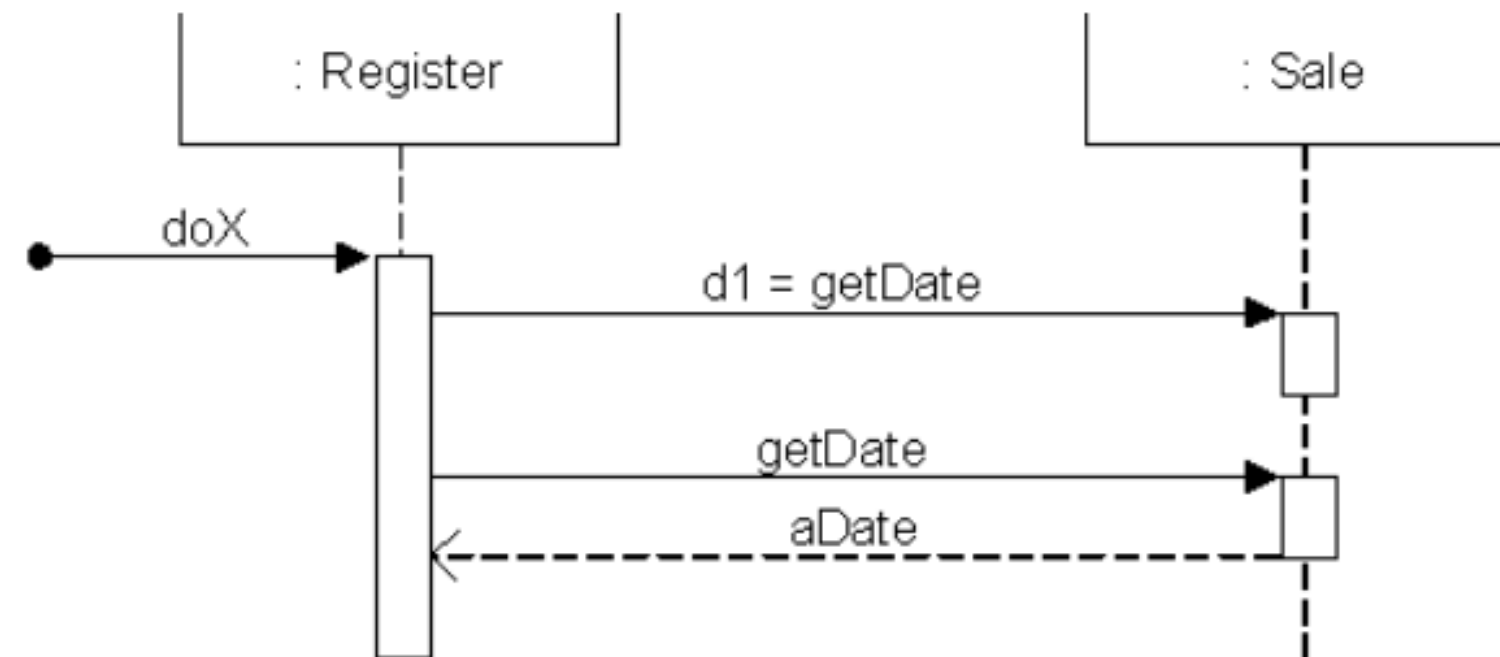
asynchronous message line
denoted by line arrowhead

return message line
denoted by dashed line

TYPES OF MESSAGES

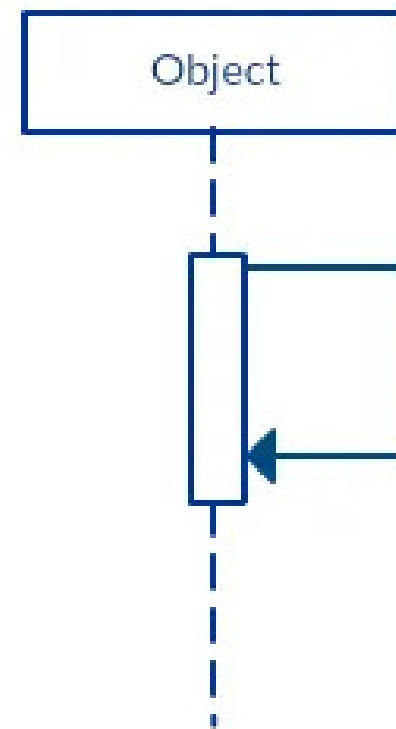
There are two ways to show return result from a message:

- Using the message syntax `returnVar=message(parameter)`
- Using a reply(or return) message line at the end of an activation bar.



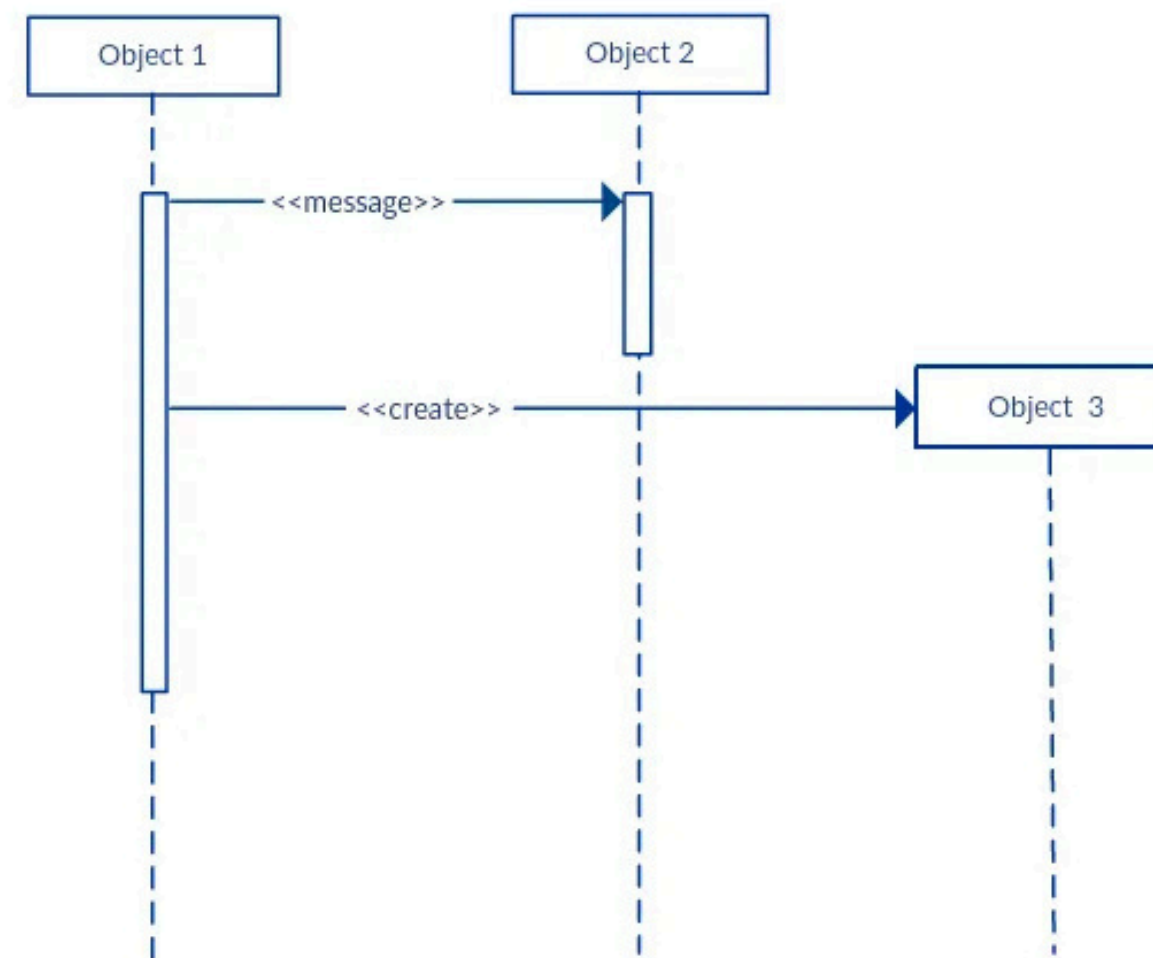
TYPES OF MESSAGES

Self Message: When an object sends a message to itself, it is called a reflexive or self message. It is indicated with a message arrow that starts and ends at the same lifeline as shown in the example below.



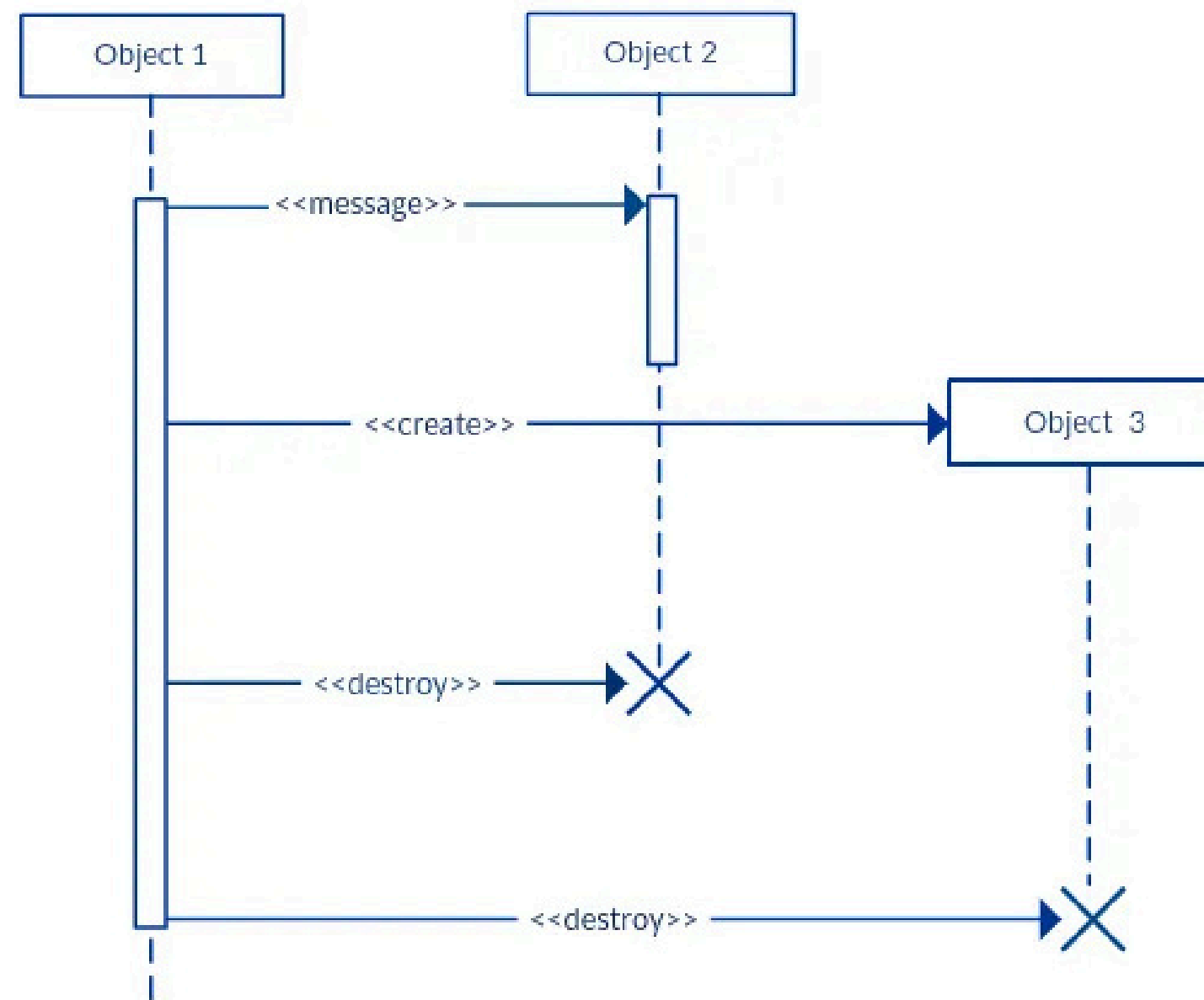
TYPES OF MESSAGES

Create Message: A message that instantiates a new object. It points directly to the new object's rectangle and is often stereotyped with <<create>>. The lifeline of the new object begins at this point.

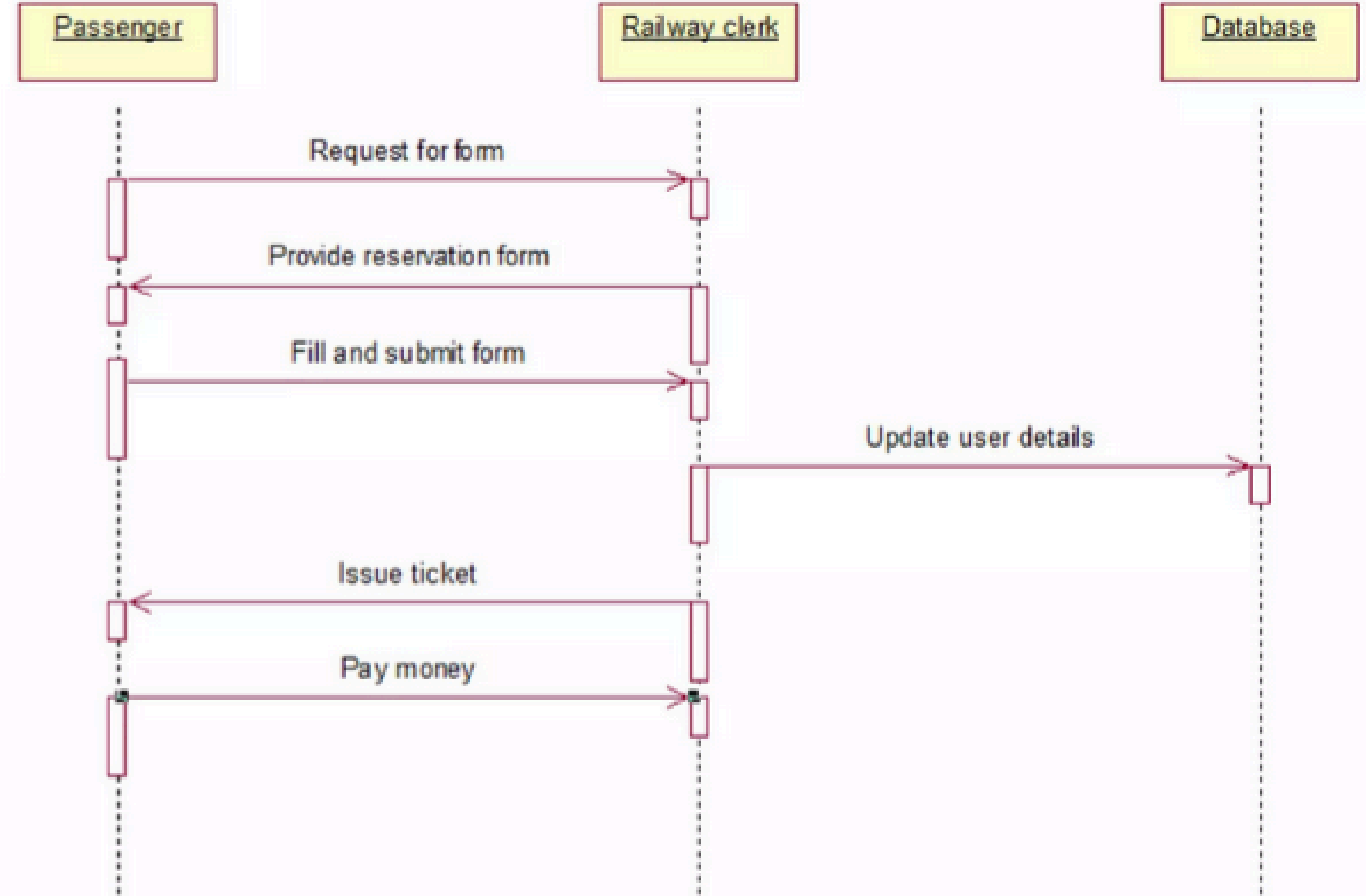


TYPES OF MESSAGES

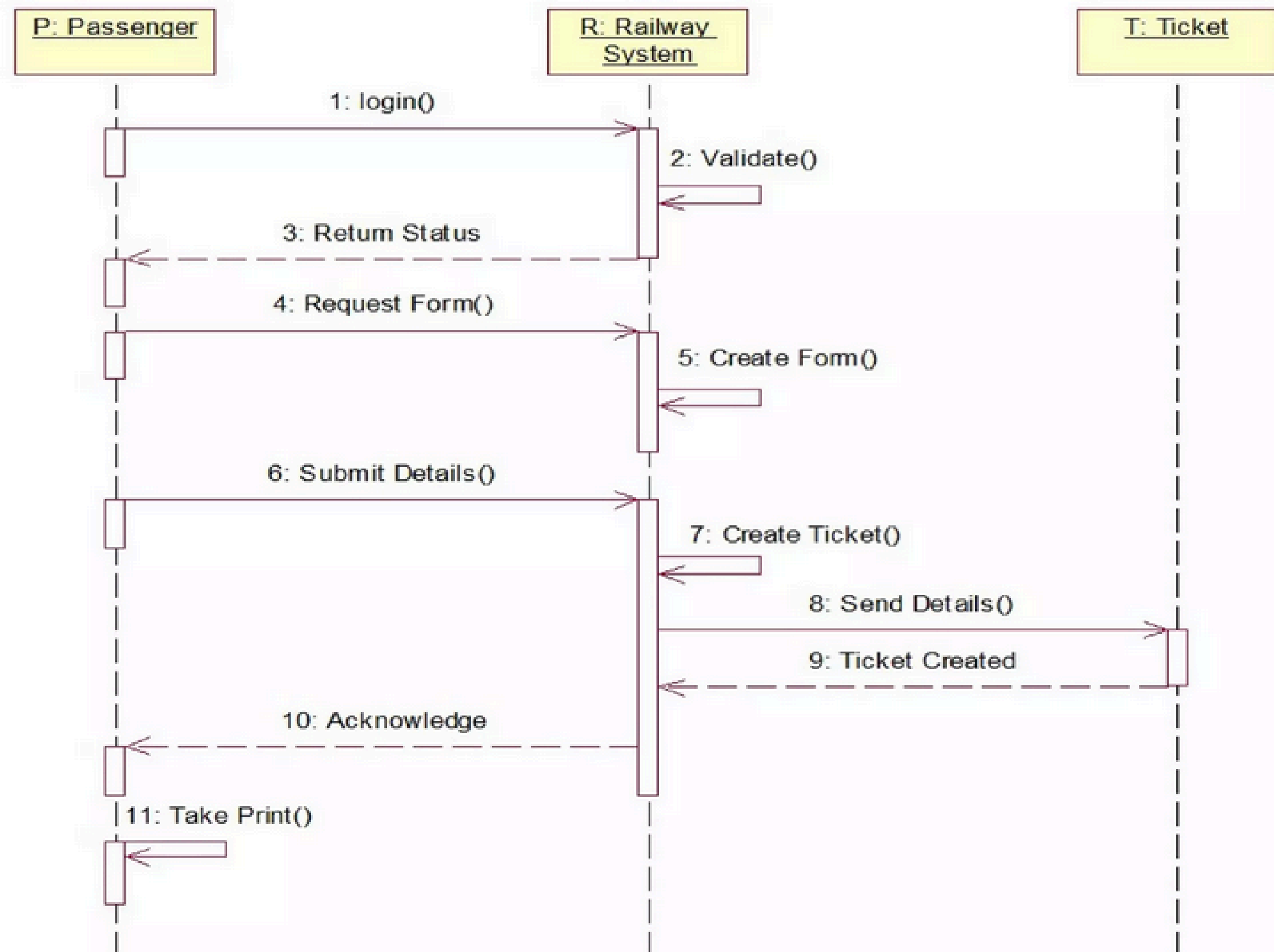
Destroy Message: An object may destroy another object via a <<destroy>> message.



RAILWAY RESERVATION EXAMPLE

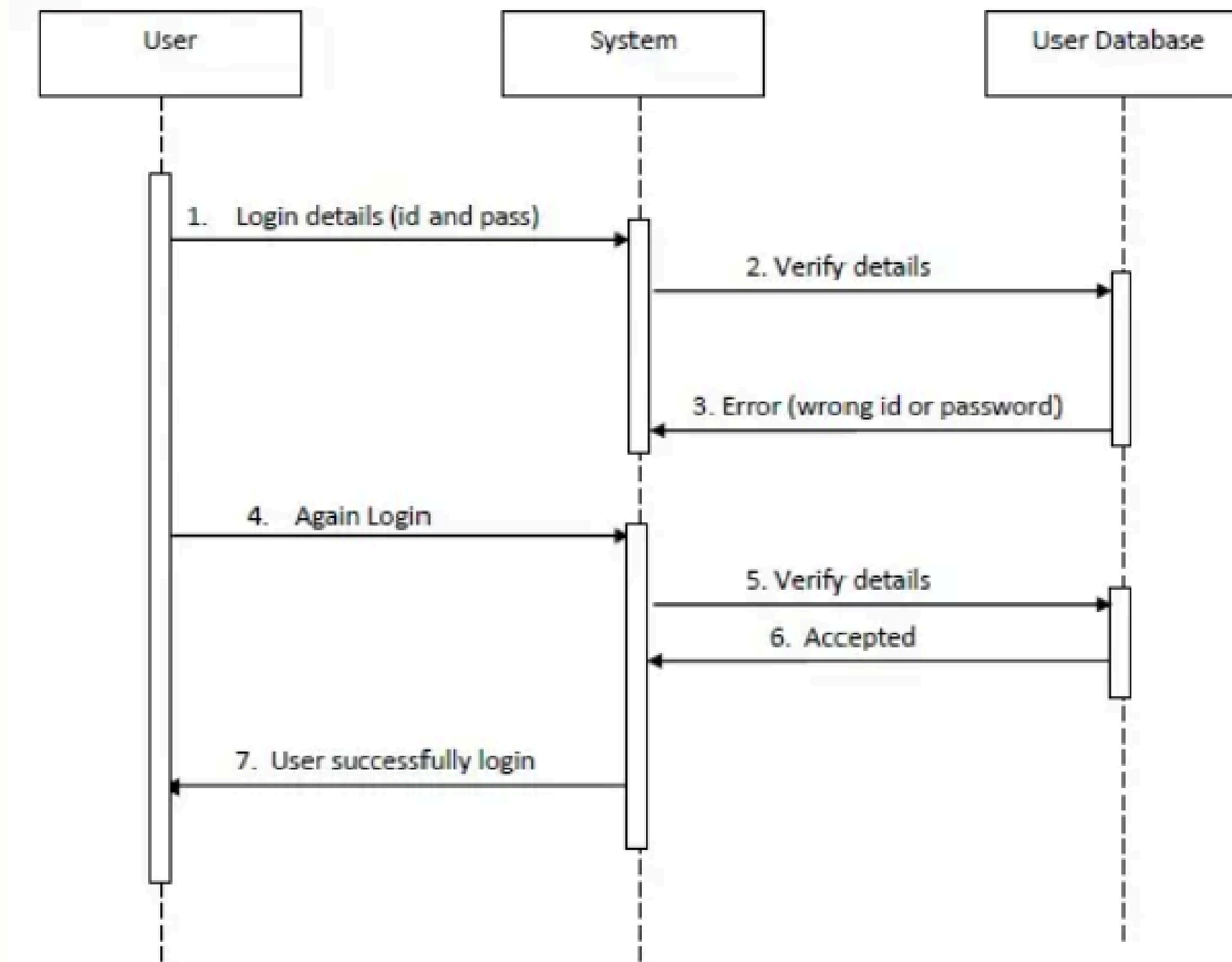


RAILWAY RESERVATION EXAMPLE



LOGIN EXAMPLE

1. LOGIN



LECTURE OUTLINE

Advance Sequence Diagram Concepts:

- Guards
- Loops
- Fragments

Further we'll explore,

- System Sequence Diagrams (SSDs)

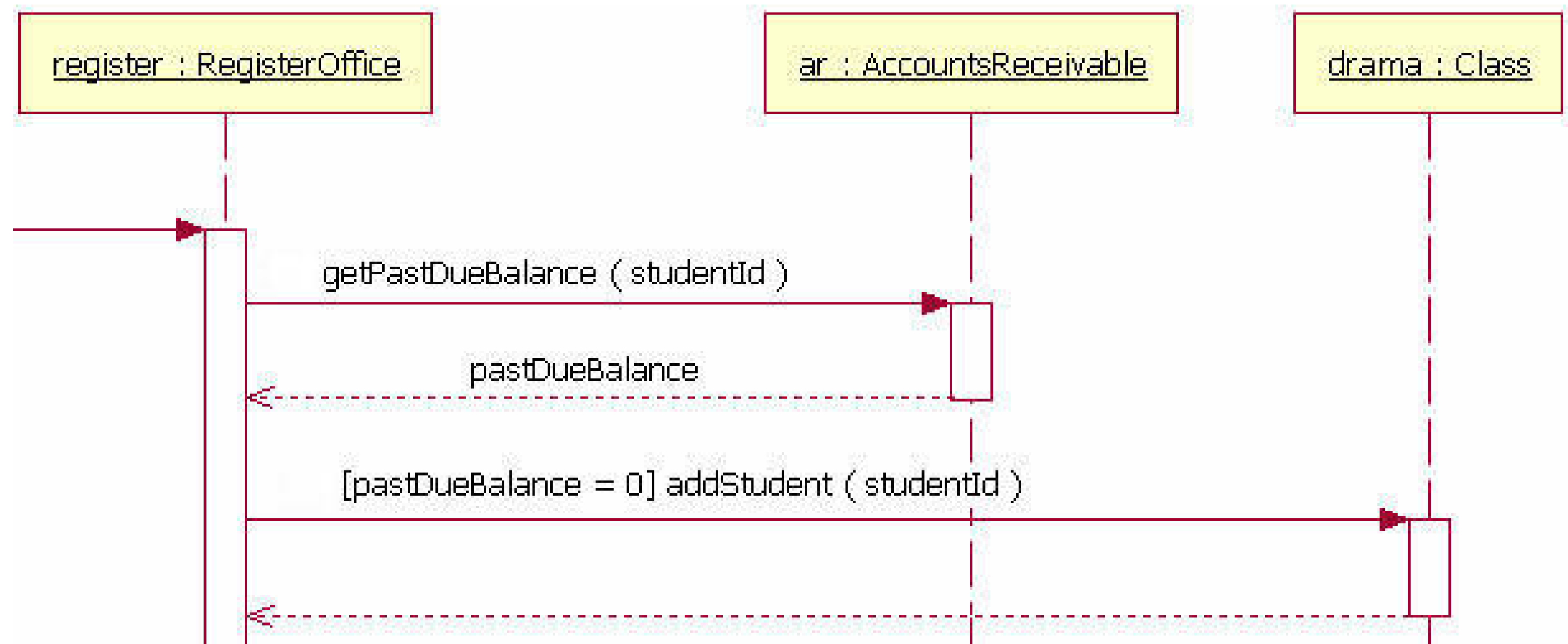
We will walk through working examples in each section

GUARDS

A guard is a condition that must be true for a message to be sent. It's like an 'if' statement in our diagram.

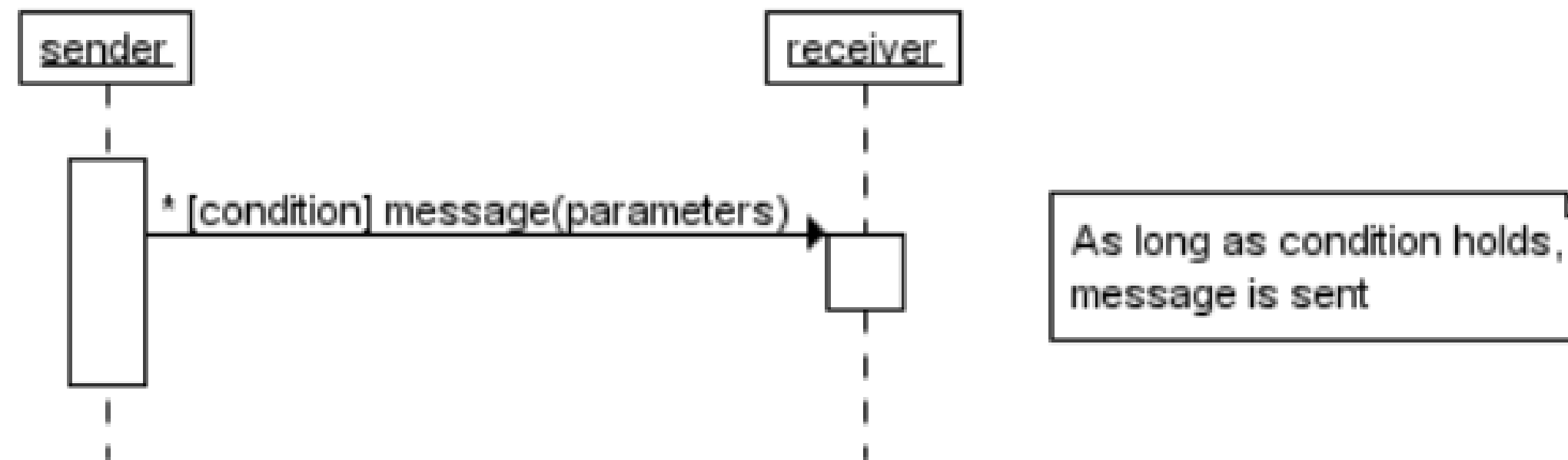
- **Notation:** The condition is written in square brackets [] before the message.
- **Example:** [pastDueBalance = 0] addStudent(studentId). This message is only sent if the student has no past-due balance.

GUARDS



REPEATED INTERACTION

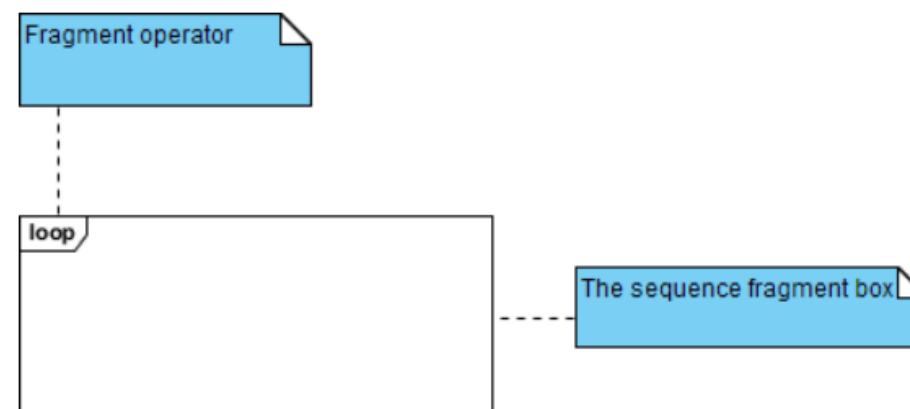
- Notation: * [condition] message(parameters)
- This means: As long as this condition is true, keep sending this message.
- Example: * [more items] enterItem(itemId) in a shopping cart.



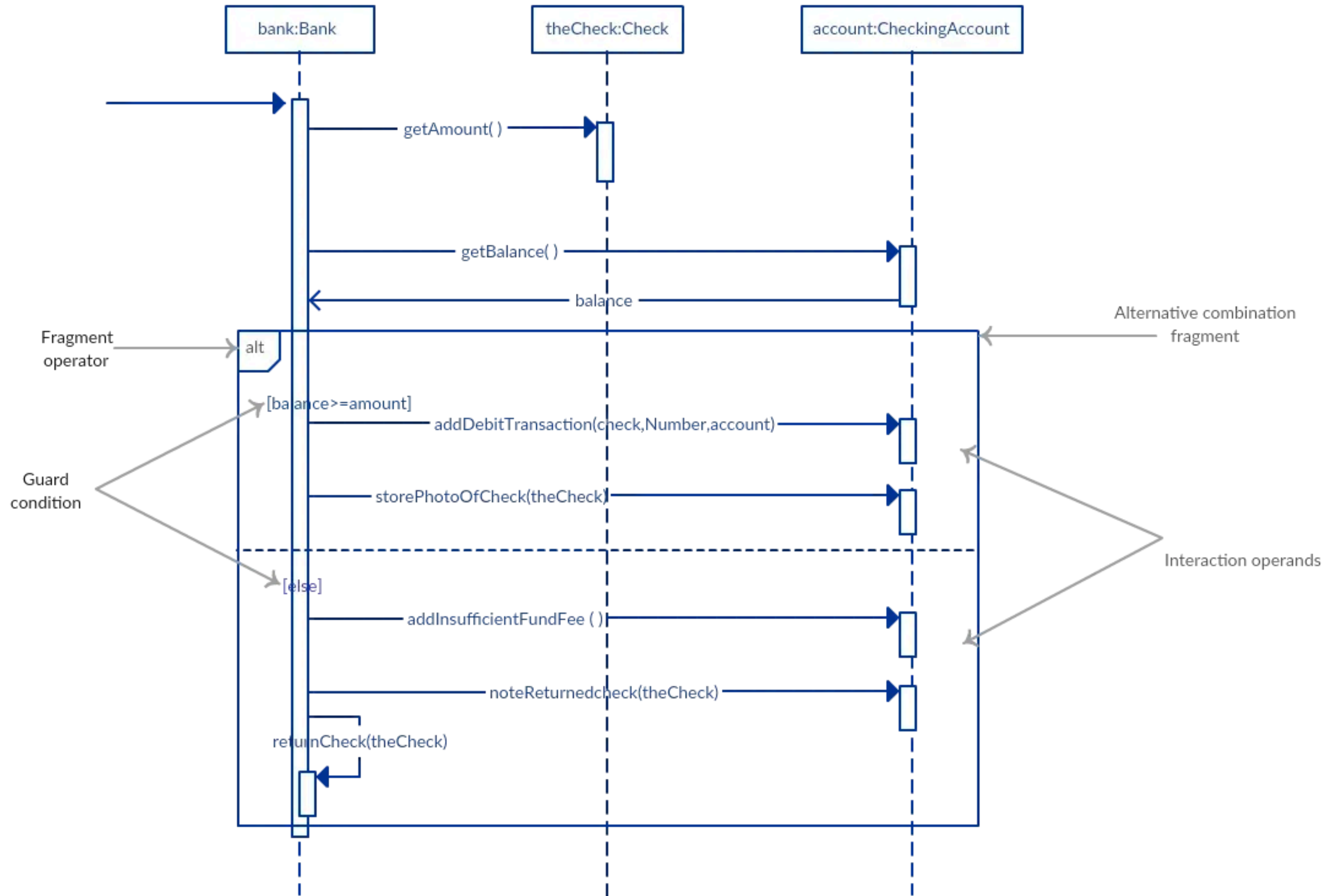
FRAGMENTS

A fragment is a named, bounded area of a sequence diagram that encapsulates a specific type of interaction logic.

- They manage complexity by grouping messages.
- They are represented by a frame with a operator in the top-left corner (like opt, alt, loop).
- This allows us to model complex control flow in a structured, readable way.



FRAGMENTS



FRAGMENTS

<u>Frame operator</u>	<u>Description</u>
opt	Optional fragment that execute if the guard is true.
alt	Alternative fragment for mutual exclusive logic expressed in the guards.
loop	Loop fragment while guard is true. Can also write loop(n) to indicate looping n times.
par	Parallel fragments that execute in parallel.
ref	Reference. This is a placeholder that says, 'The details for this part are in another diagram.' This is crucial for breaking down large diagrams.



FRAGMENTS

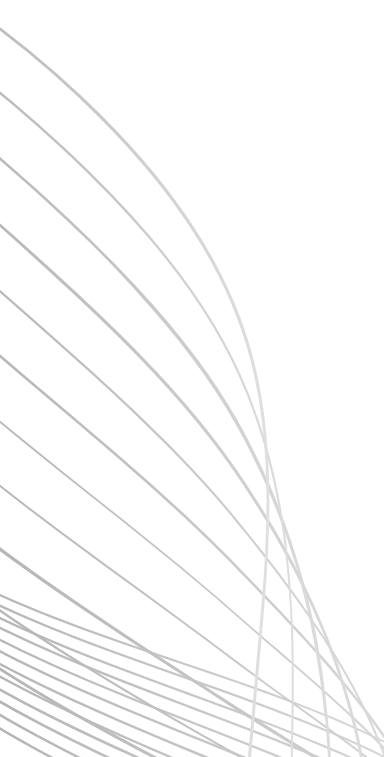
alt: Alternative. Like an if/else if/else block. Only one segment executes.

opt: Optional. A simpler alt with only one branch. Like a simple if with no else.

loop: Loop. The segment repeats while a guard condition is true.

par: Parallel. The segments inside run concurrently.

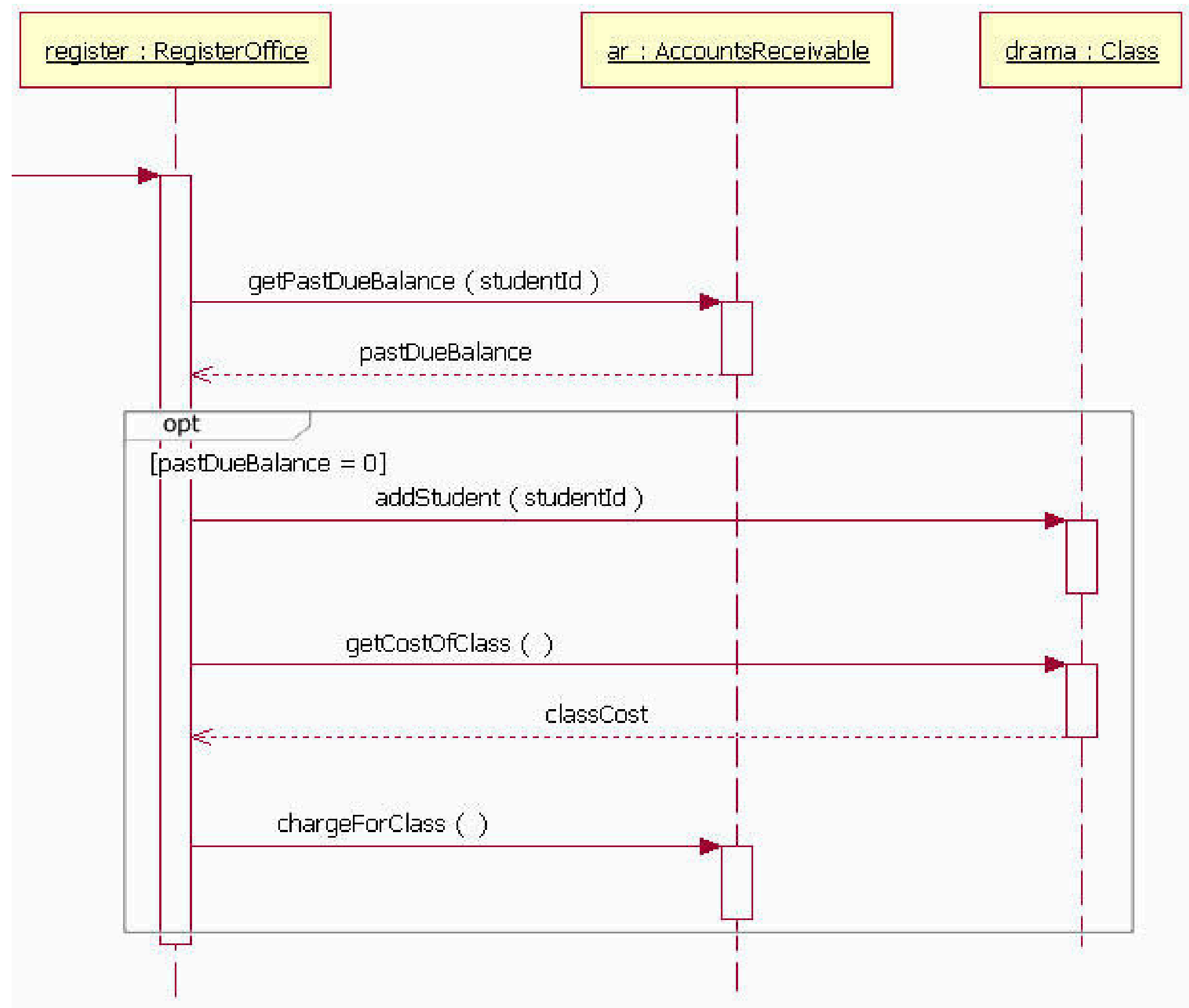
ref: Reference. This is a placeholder that says, 'The details for this part are in another diagram.' This is crucial for breaking down large diagrams.



OPTION: OPT

- The option combination fragment is used to model a sequence that, given a certain condition, will occur; otherwise, the sequence does not occur.
- An option is used to model a simple "if then" statement
 - Example, if there are fewer than five donuts on the shelf, then make two dozen more donuts.

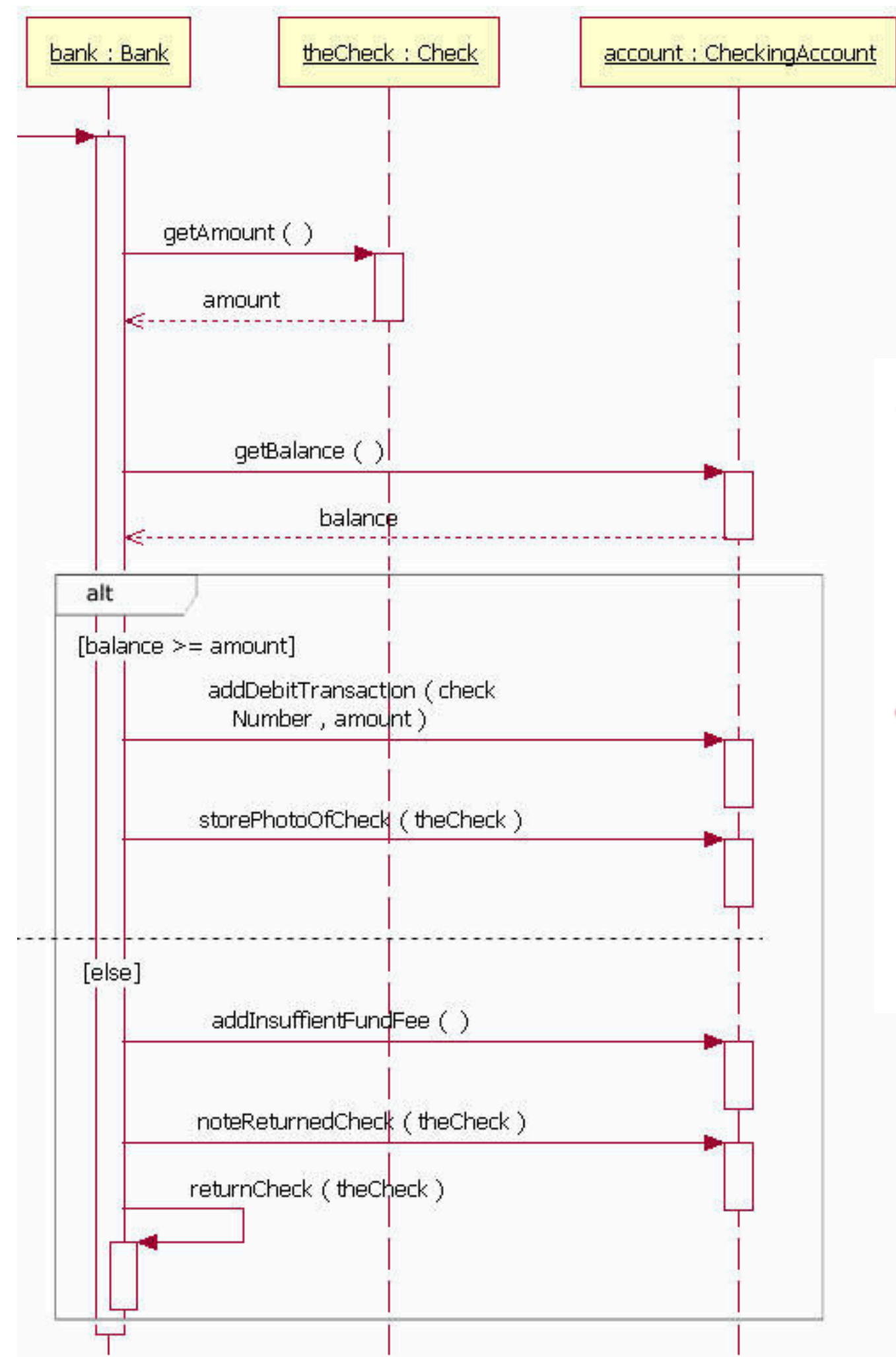
```
if(pastDueBalance == 0)
{
    /*
     Set of messages to be executed
    */
}
```





ALTERNATIVE: ALT

- The fragment is divided by a dashed line. Each partition has a guard.
- For example, in a login sequence: [valid credentials] shows the success messages, and [invalid credentials] shows the error message. Only one of these alternatives will occur.



```

if (balance >= amount)
{
    /*
    Proceed with transaction
    */
}
else
{
    /*
    Insufficient Balance
    */
}
  
```

MODELING 'SEARCH BOOK' USE CASE

Search Book : Use Case

- Main scenario -

The Customer specifies an author on the Search Page and then presses the Search button.

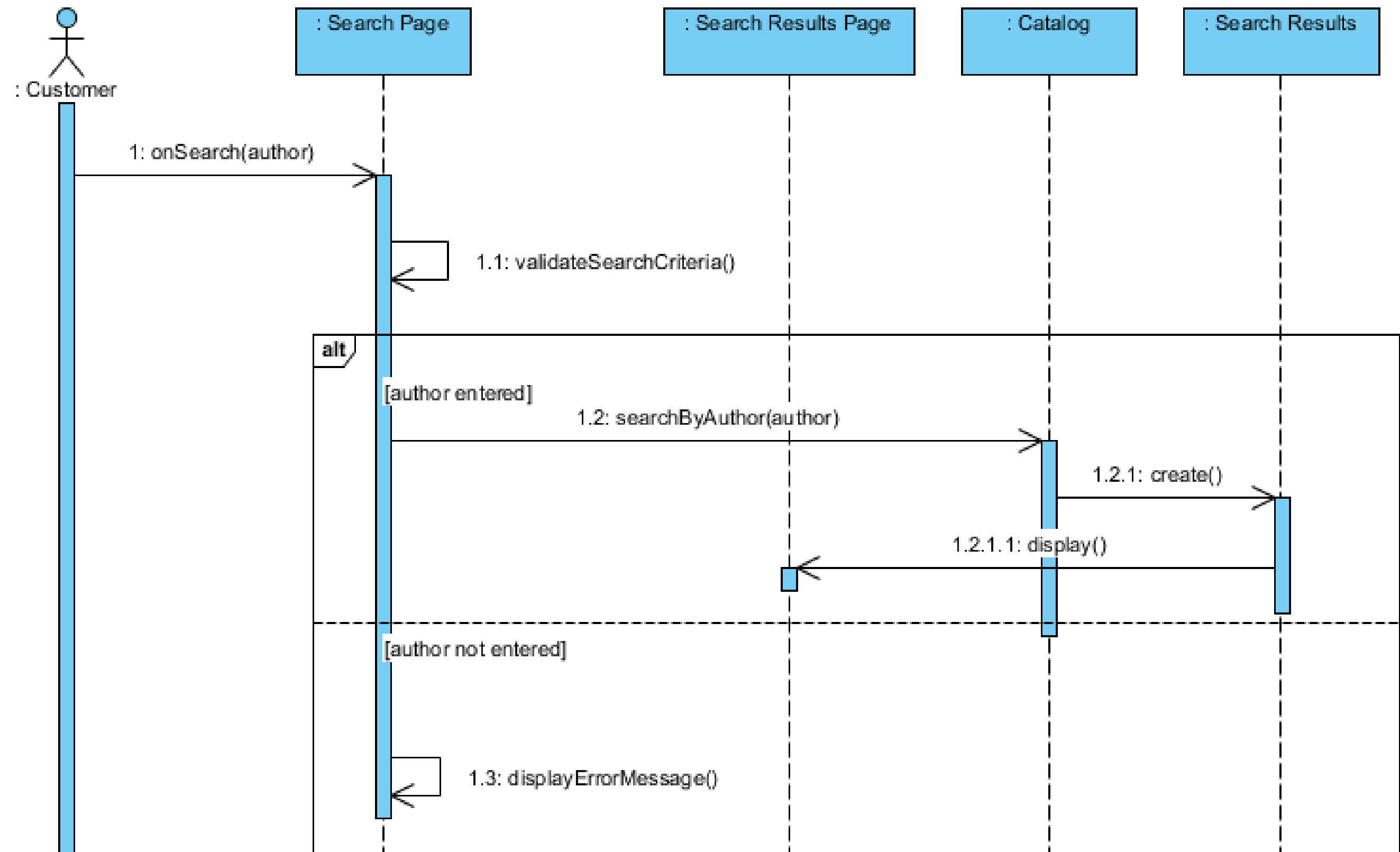
The system validates the Customer's search criteria.

If author is entered, the System searches the Catalog for books associated with the specified author.

When the search is complete, the system displays the search results on the Search Results page.

- Alternate path -

If the Customer did not enter the name of an author before pressing the Search button, the System displays an error message

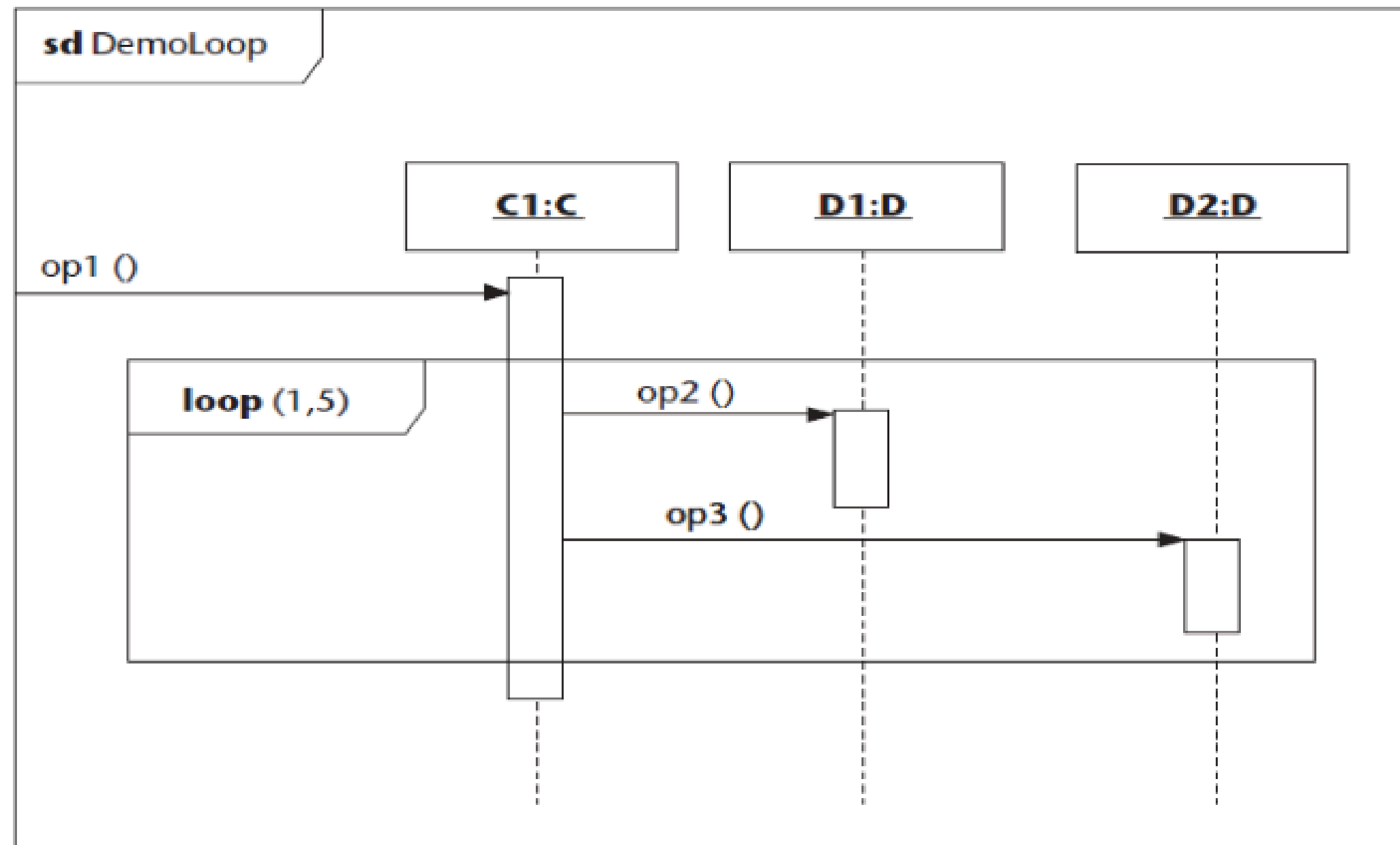


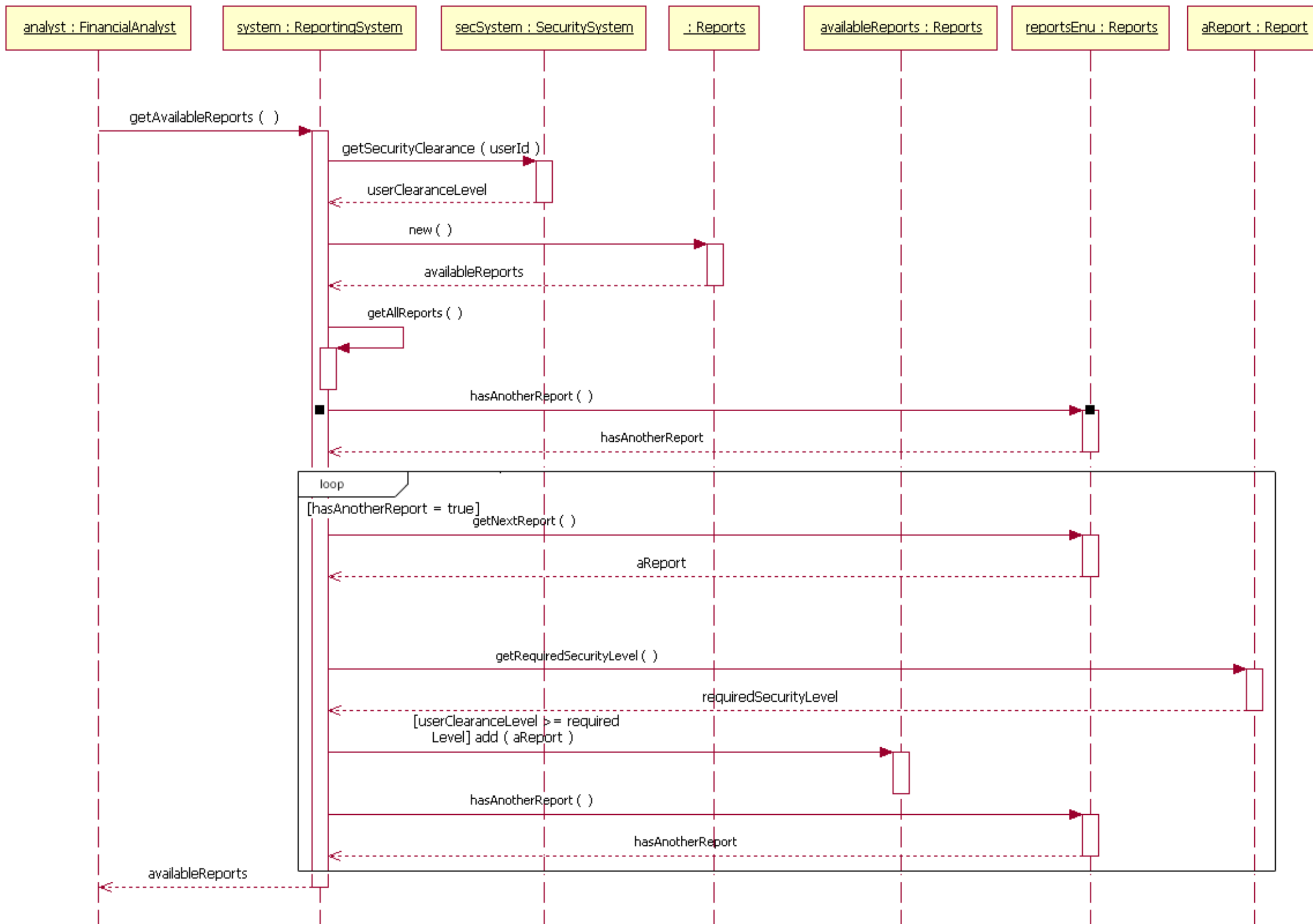
LOOP

Occasionally you will need to model a repetitive sequence, for modeling repetition the loop fragment is used. The operator is loop, and in the frame, you specify the loop guard.

- **Notation:** loop (min, max) or just loop [condition].
- **Example:** loop (1, 5) means 'repeat this block between 1 and 5 times.' In the diagram, the messages op2 and op3 are repeated five times.

LOOP





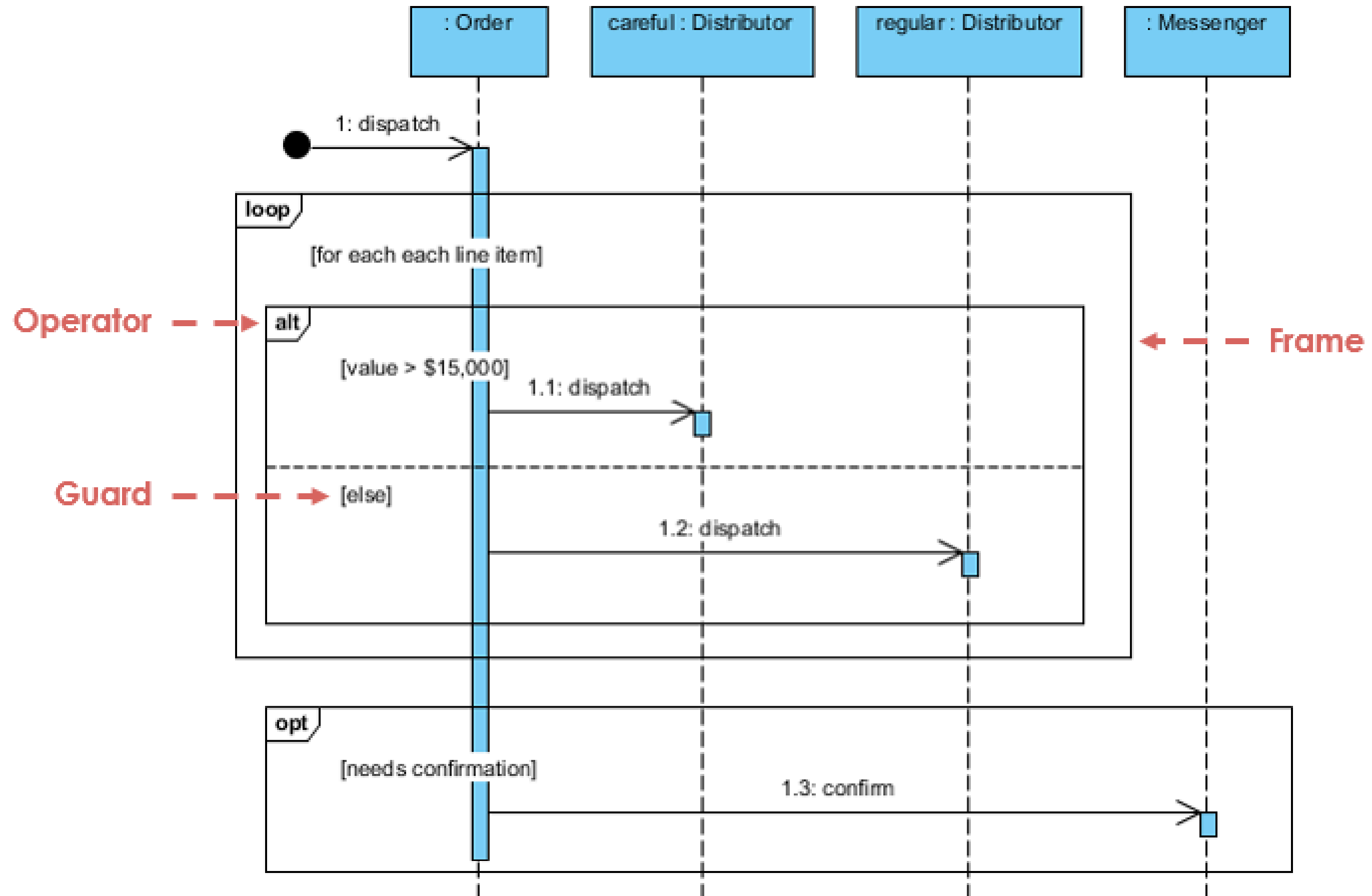
LOOP

EXAMPLE

Consider an order dispatch system

- We have a loop fragment for 'each line item' in the order.
- Inside the loop, we have an alt fragment. The condition checks the item's value.
 - [value > \$10000] -> send to express dispatcher.
 - [else] -> send to regular dispatcher.
- After the loop, we have an opt fragment: [needsConfirmation] confirm. This message is only sent if the customer asked for confirmation.

EXAMPLE

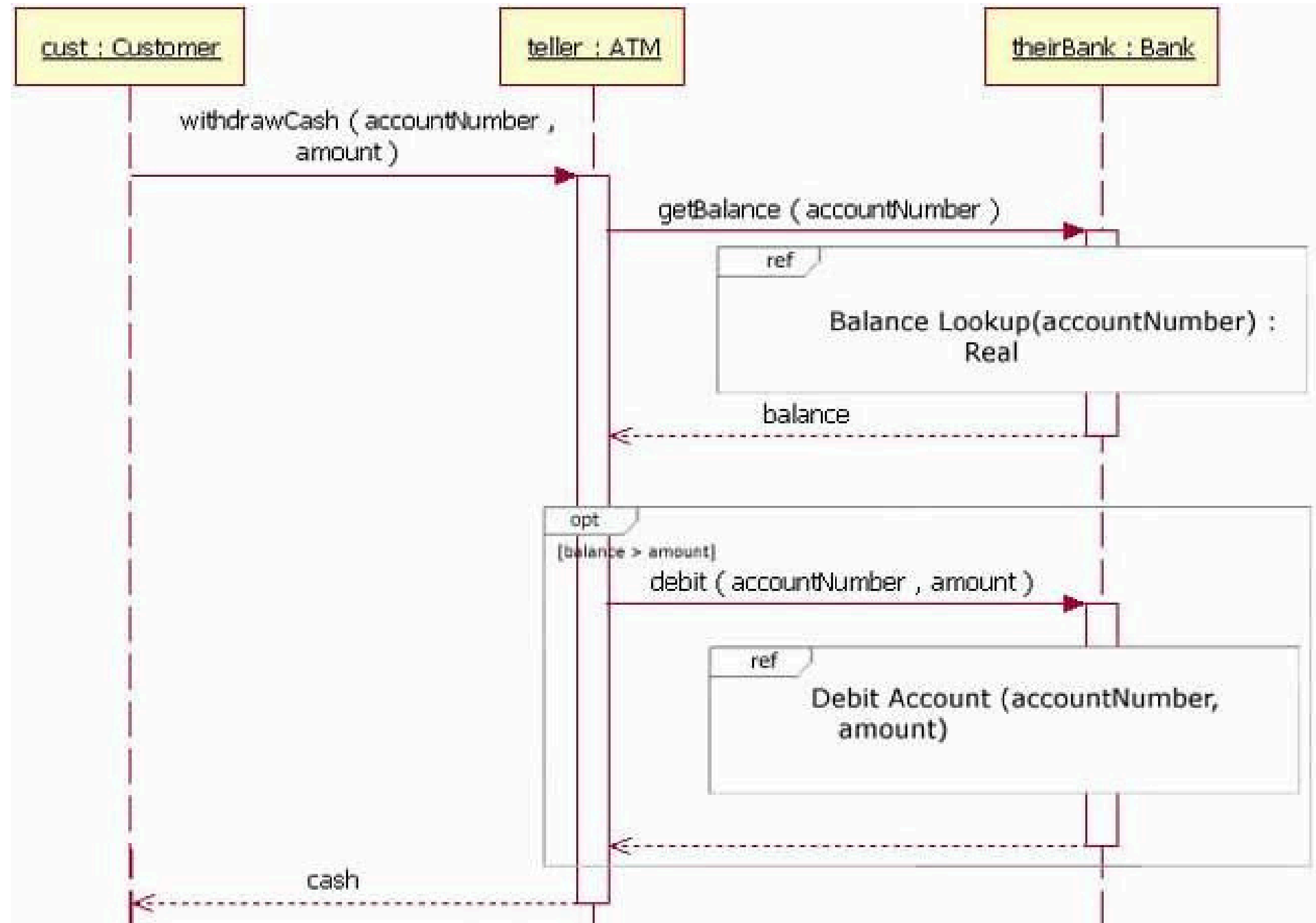


REFERENCE FRAME: REF

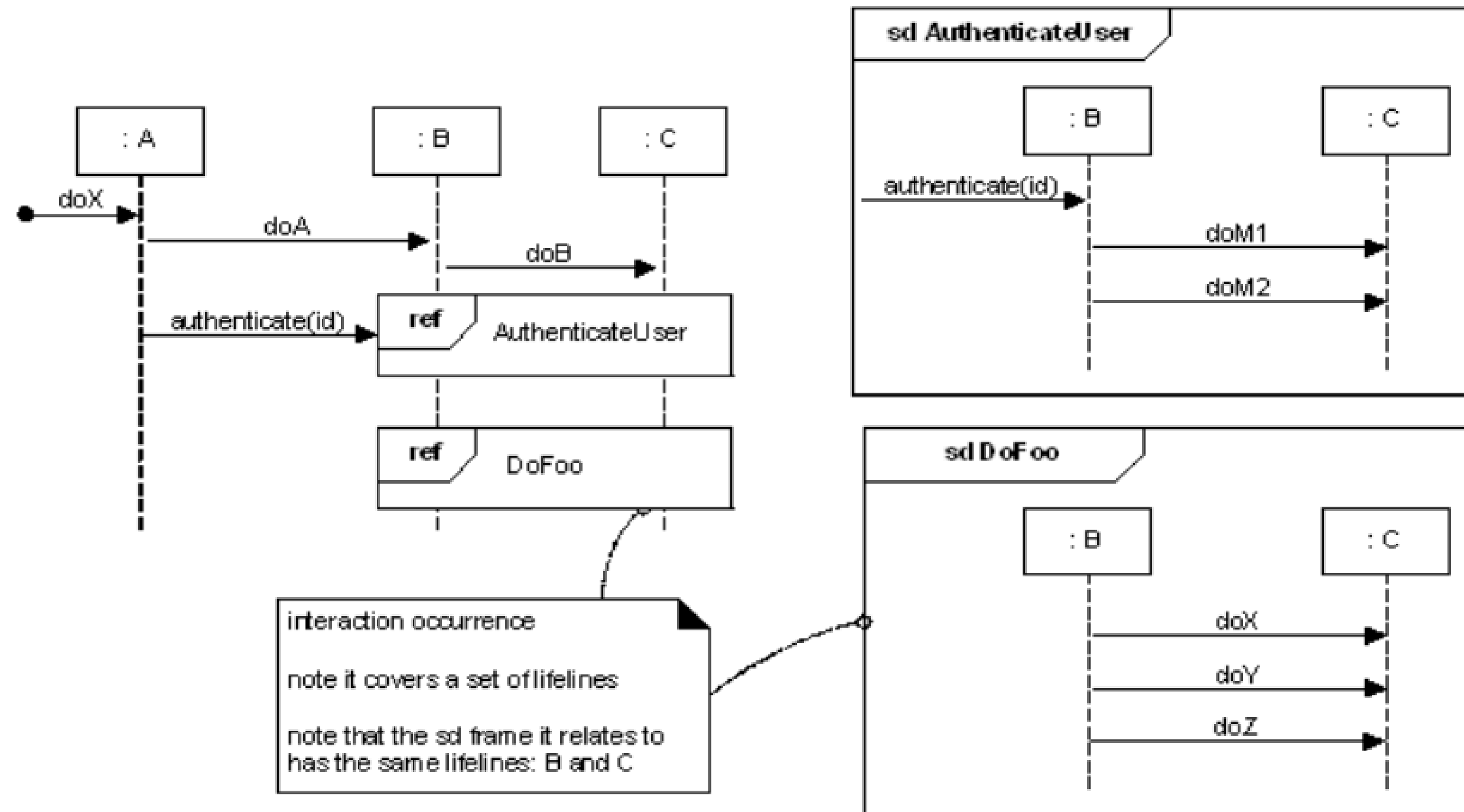
This is for managing size. Imagine your sequence diagram is getting too long and complicated. You can break it up.

- You can create a separate sequence diagram for a sub-process (e.g., 'Authenticate User').
- In your main diagram, you simply use a ref fragment, point to the lifelines involved, and write the name of the other diagram inside (e.g., `ref AuthenticateUser`).
- This is like calling a function in code, you don't see all the details, just the call.

REFERENCE FRAME: REF



REFERENCE FRAME: REF

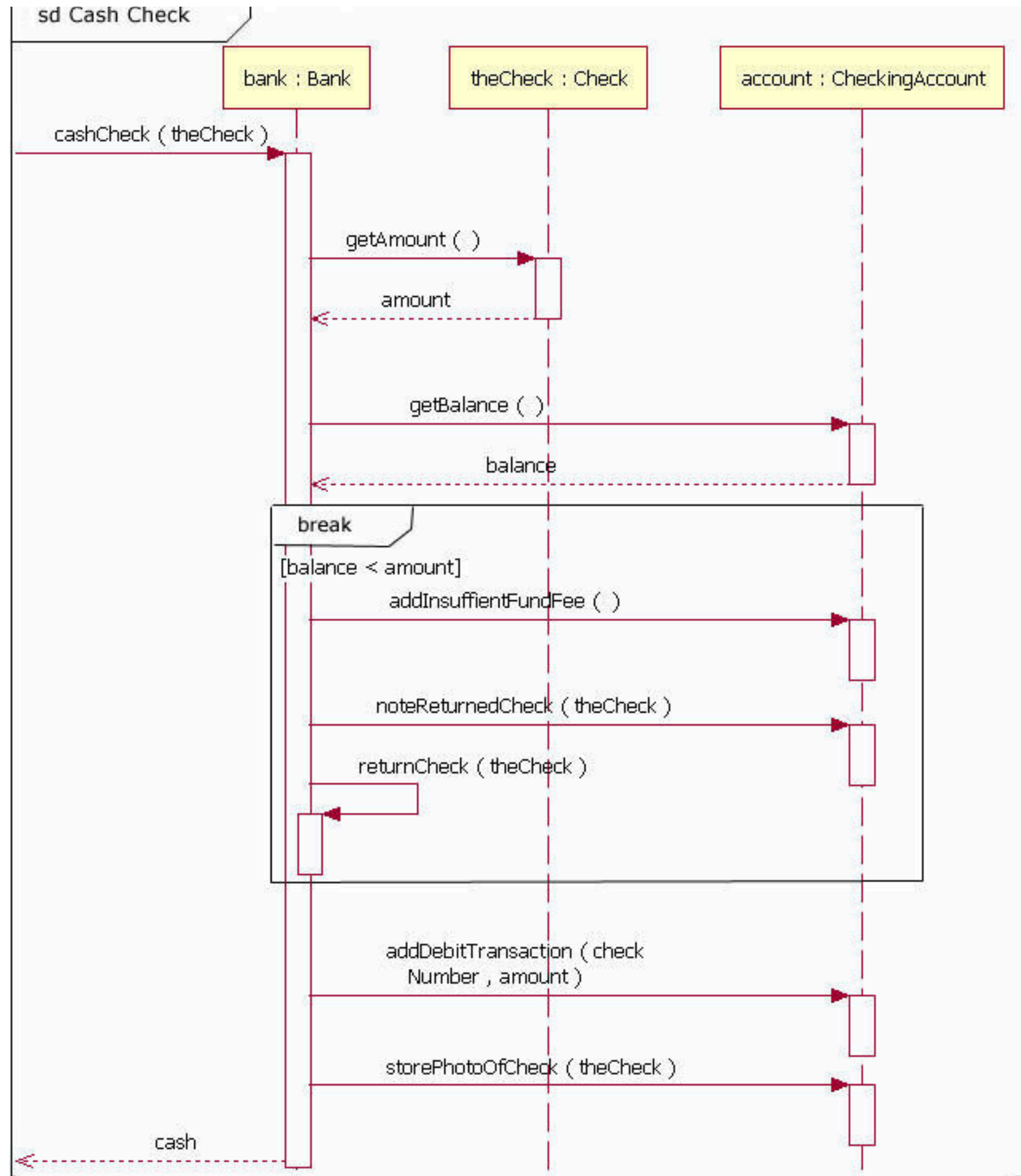


BREAK

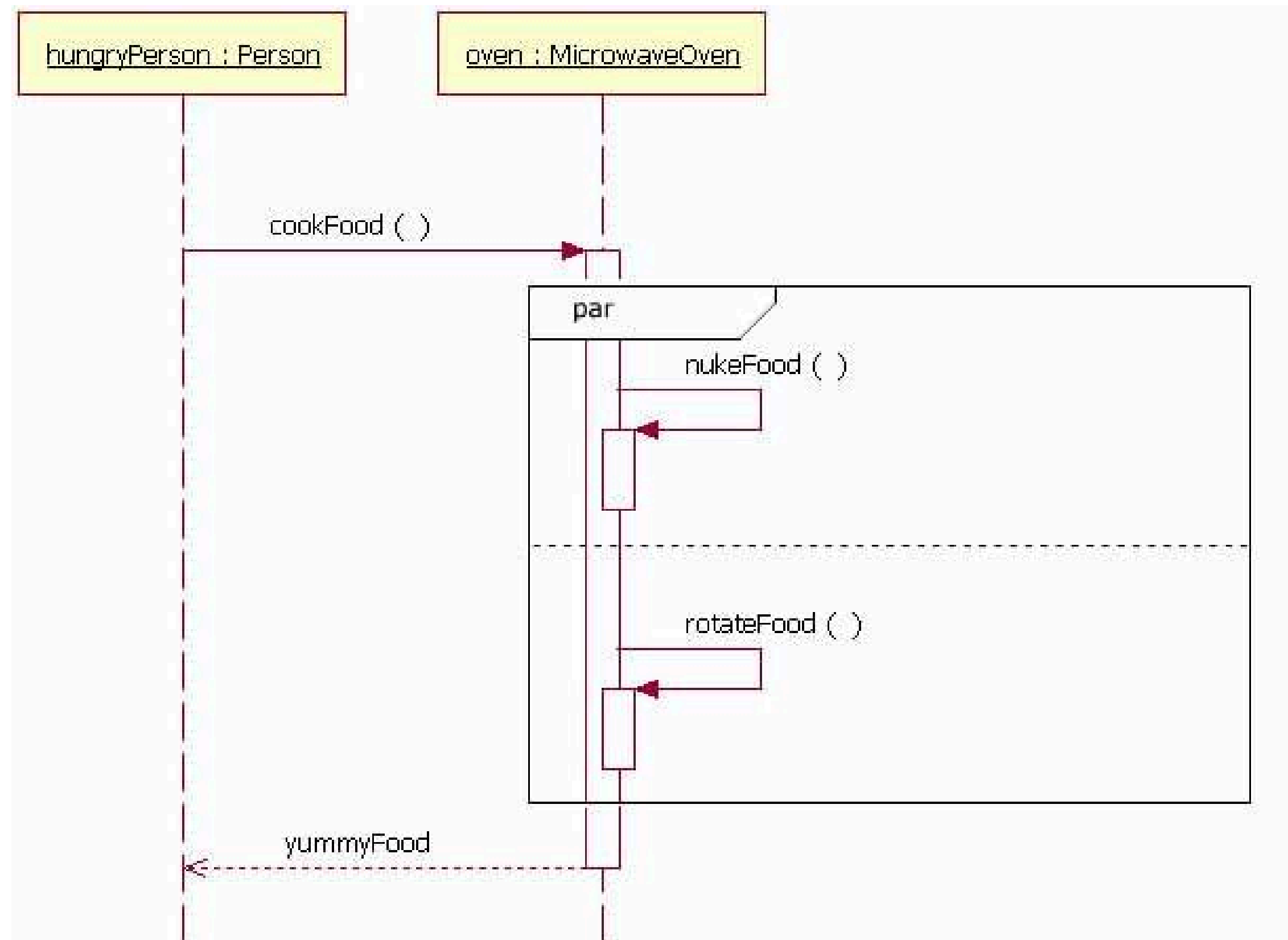
The break combined fragment is almost identical in every way to the option combined fragment, with two exceptions. First, a break's frame has a namebox with the text "break" instead of "option." Second, when a break combined fragment's message is to be executed, the enclosing interaction's remainder messages will not be executed because the sequence breaks out of the enclosing interaction.

similar to **break keyword** in programming languages like C, C++ or Java

BREAK



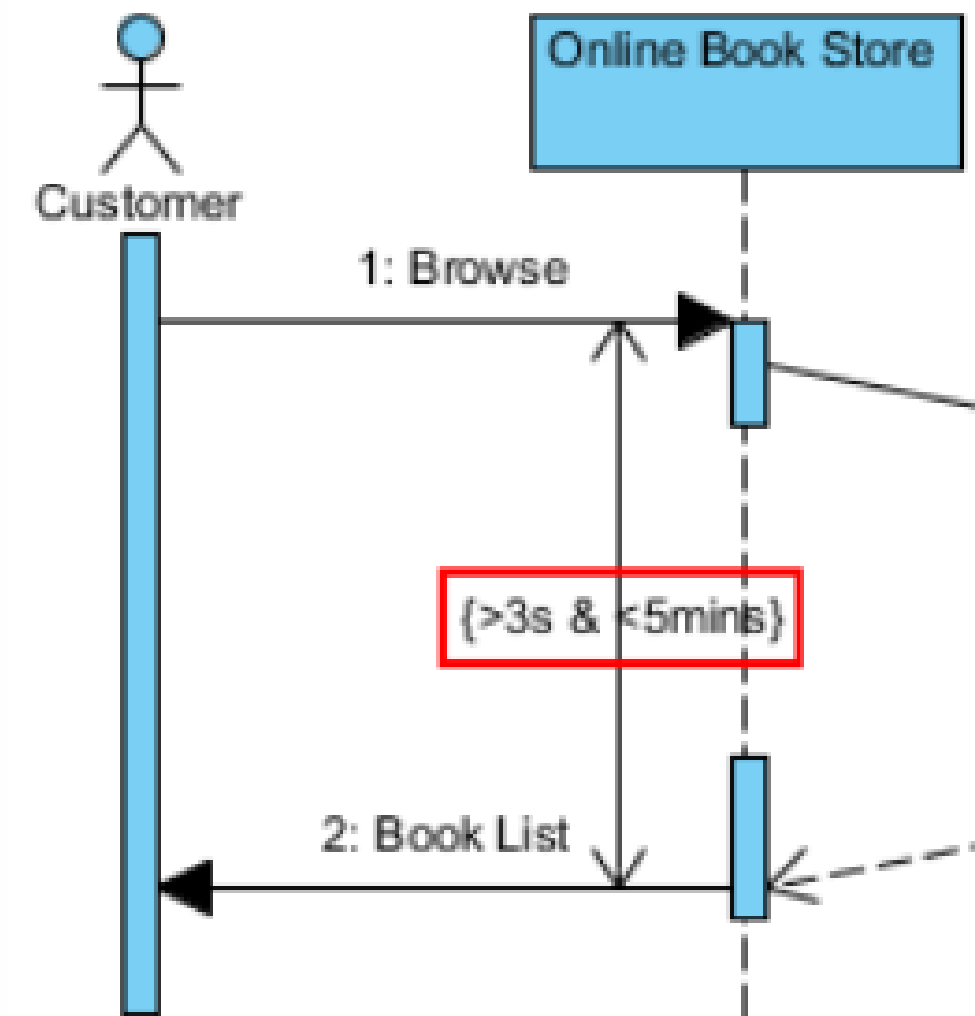
PARALLEL: PAR



SPECIFYING TIMING REQUIREMENTS

Timing Constraints: For real-time or performance critical systems, you can specify how long an operation should take.

- **Notation:** You write the constraint inside curly braces {} on the message or between two events.
- **Example:** {>3s & <5mins} between a Browse message and the Book List reply. This means the system must respond within that time window.





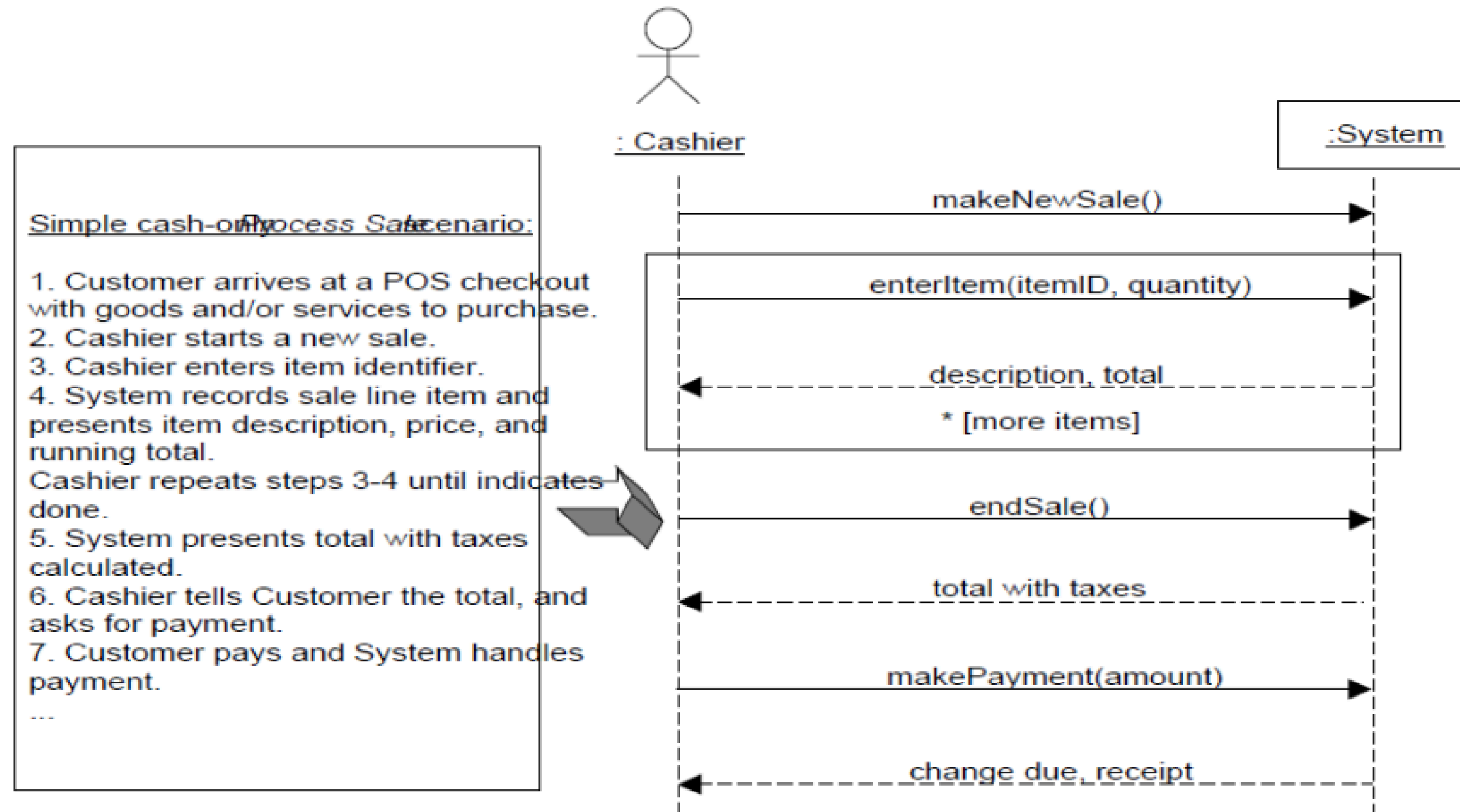
HOW TO DRAW GOOD SEQUENCE DIAGRAMS

- **Decide on Context:** Which scenario or use case are you modeling?
- **Identify Structural Elements:** Find the objects, draw the lifelines.
- **Model the Interaction:** Add messages, activations.
- **Elaborate:** Use fragments, guards, and timing constraints to add detail and complexity

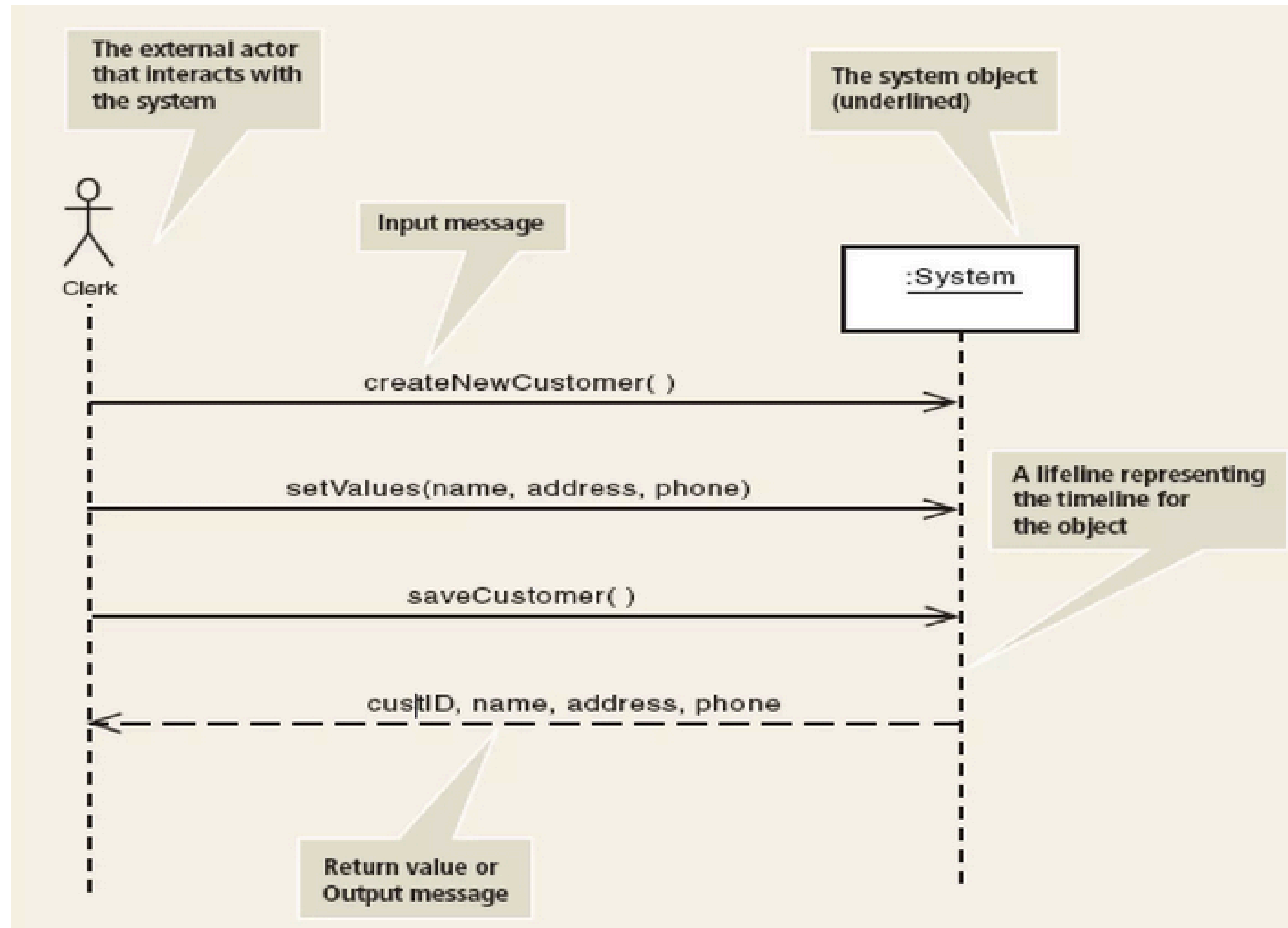
SYSTEM SEQUENCE DIAGRAM

- A System Sequence Diagram (SSD) is a specific sub-type of a sequence diagram.
- Focus: It shows the interaction between an Actor and the System as a whole.
- Elements: The only lifelines are the Actor(s) and the System (represented as a black box).
- Purpose: It captures the system events, the requests the actor makes to the system and the system's responses.
- When to use: Derived directly from use cases, especially the main success scenario.
- It's a high level view that helps identify the major input and output operations of the system.

SYSTEM SEQUENCE DIAGRAM (SSD)



SYSTEM SEQUENCE DIAGRAM (SSD)





COLLABORATION DIAGRAM

Lecture Objectives: while Collaboration Diagrams show the same information as Sequence Diagram, they present it through a completely different lens. Our goal for this section is to understand their **semantics**, learn their **unique notation**, and see how they compare to what we already know.



COLLABORATION DIAGRAM

What is a Collaboration Diagram?

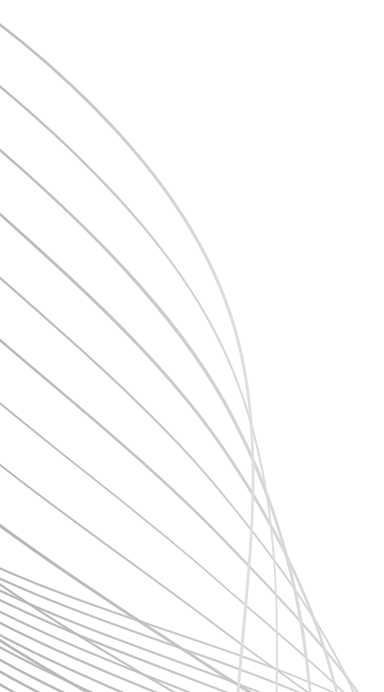
- It is a member of the Behavioral group of diagrams, just like sequence diagrams.
- Its primary purpose is to capture the dynamic behavior of a system—how objects interact at runtime.
- But its superpower is visualizing the relationship between objects that are collaborating to perform a particular task.

We can think of it as a map of object interactions

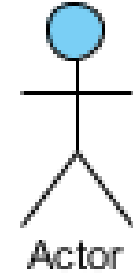


ELEMENTS OF A COLLABORATION DIAGRAM

Every collaboration diagram is built from three core elements:

1. **Objects** - The 'who' in the interaction.
 2. **Links** - The 'how' they are connected.
 3. **Messages** - The 'what' they are saying to each other.
- 

COLLABORATION DIAGRAM NOTATIONS

Actor	
Object	
Association/Links	
Message	

OBJECTS

Objects are represented exactly as they are in sequence diagrams and class diagrams:
a rectangle with the objectName : ClassName format. The object name is optional, but
the class name is mandatory.

- Sale - This represents the Sale class itself (less common).
- :Sale - This is an anonymous instance of the Sale class.
- s1:Sale - This is a named instance (s1) of the Sale class. This is the clearest form.

Sale

:Sale

s1: Sale

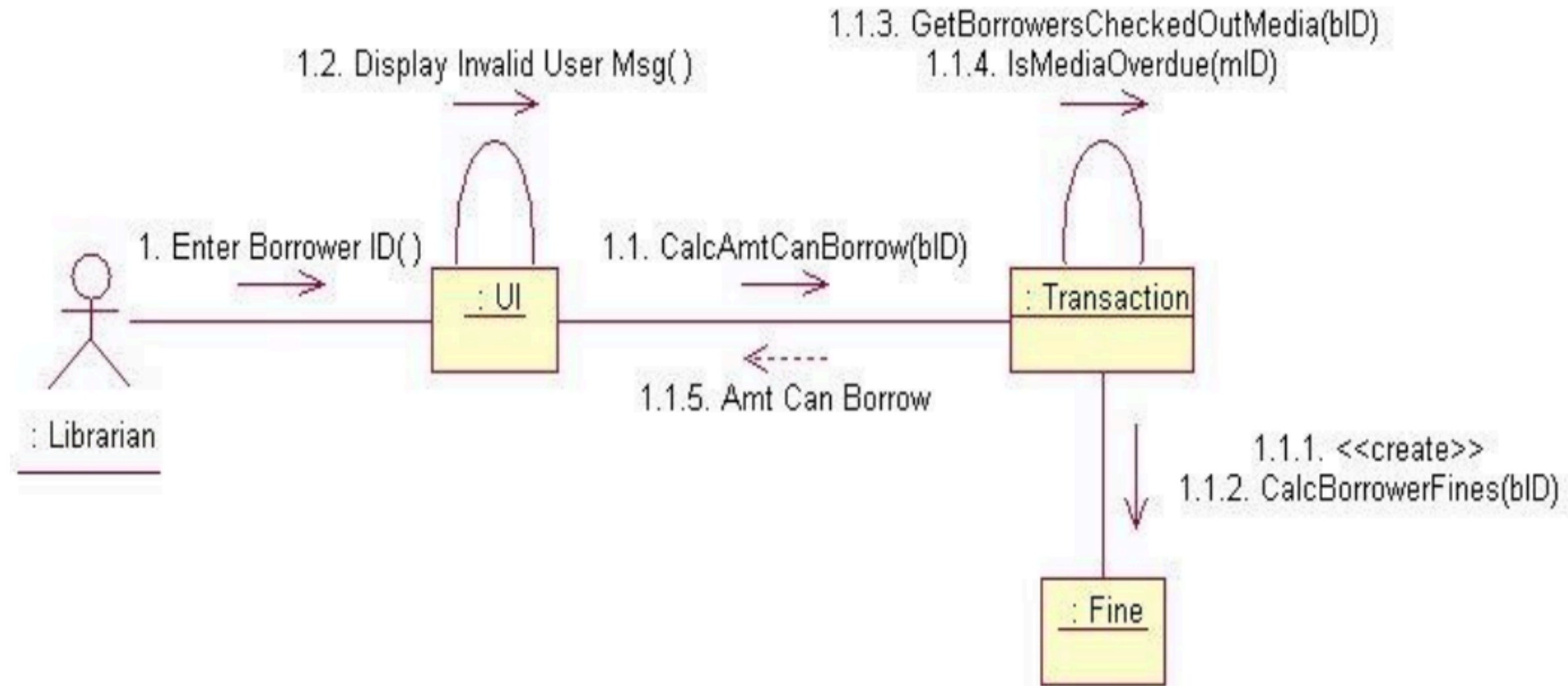


CLIENT VS. SUPPLIER OBJECTS

- The Client object is the one that sends a message. It initiates the request.
- The Supplier object is the one that receives the message. It provides the service or operation.

In any interaction, an object can play the role of client, supplier, or both.

CLIENT VS. SUPPLIER OBJECTS

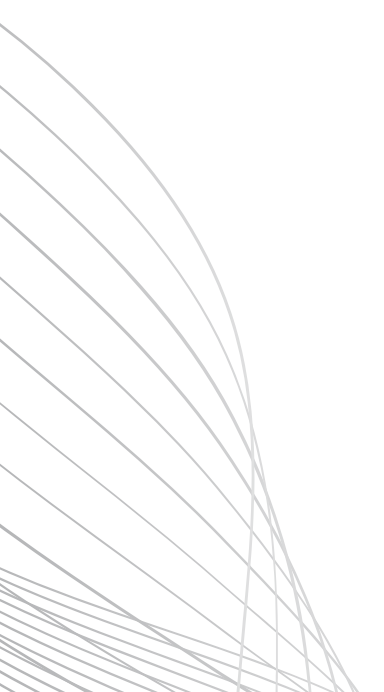


In the example, the UI object is a client when it sends a message to the Transaction object. The Transaction object is the supplier in that interaction. In turn, the Transaction object becomes a client when it sends a message to the Fine object.

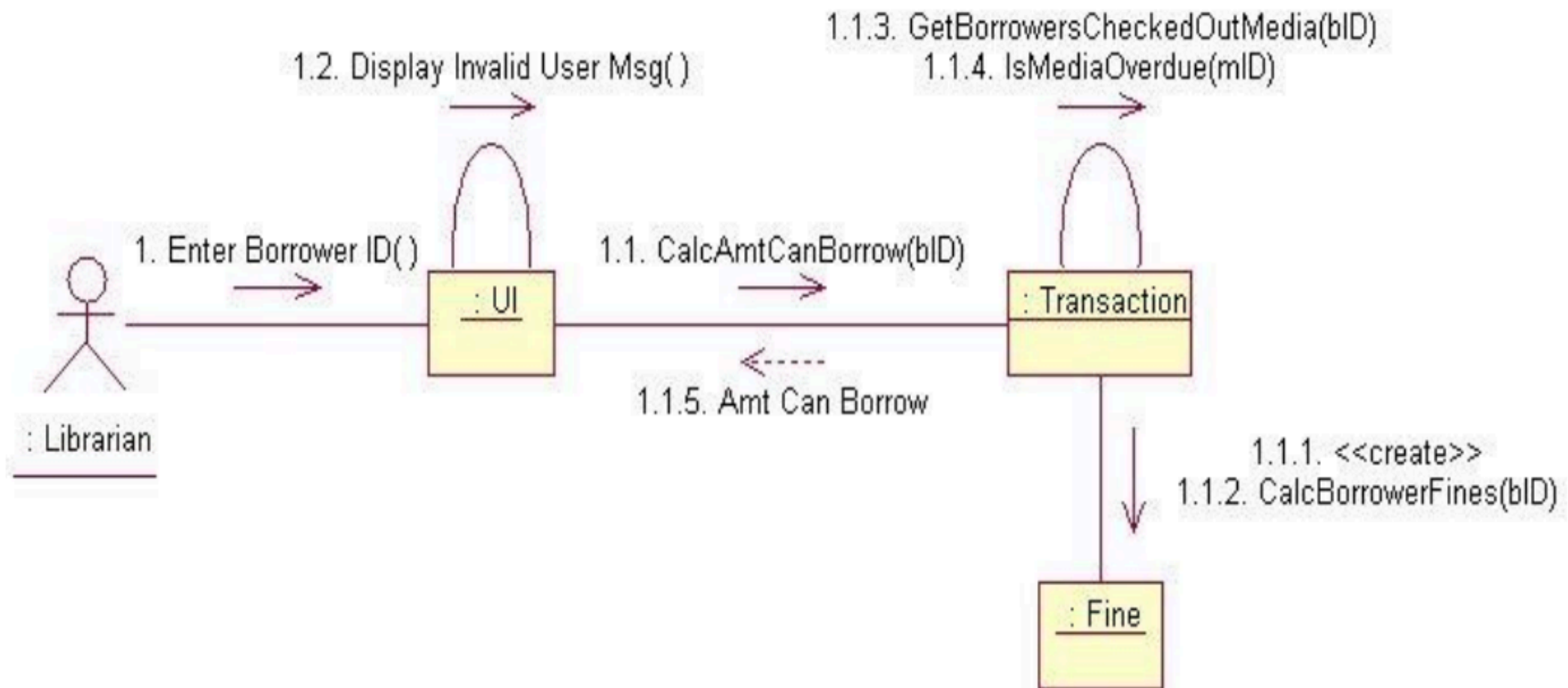


LINKS

Links are the straight lines you draw between objects. They are the visual representation of an association, a pathway that allows objects to communicate.

- A link signifies that the objects can 'see' each other; one can send a message to the other.
 - A single link can support multiple messages going back and forth.
 - If an object sends a message to itself, the link is shown as a loop connecting back to the same object.
- 

LINKS



MESSAGES

Messages are shown as arrows pointing from the client object to the supplier object along a link.

The crucial difference from sequence diagrams is how we represent sequence and nesting.

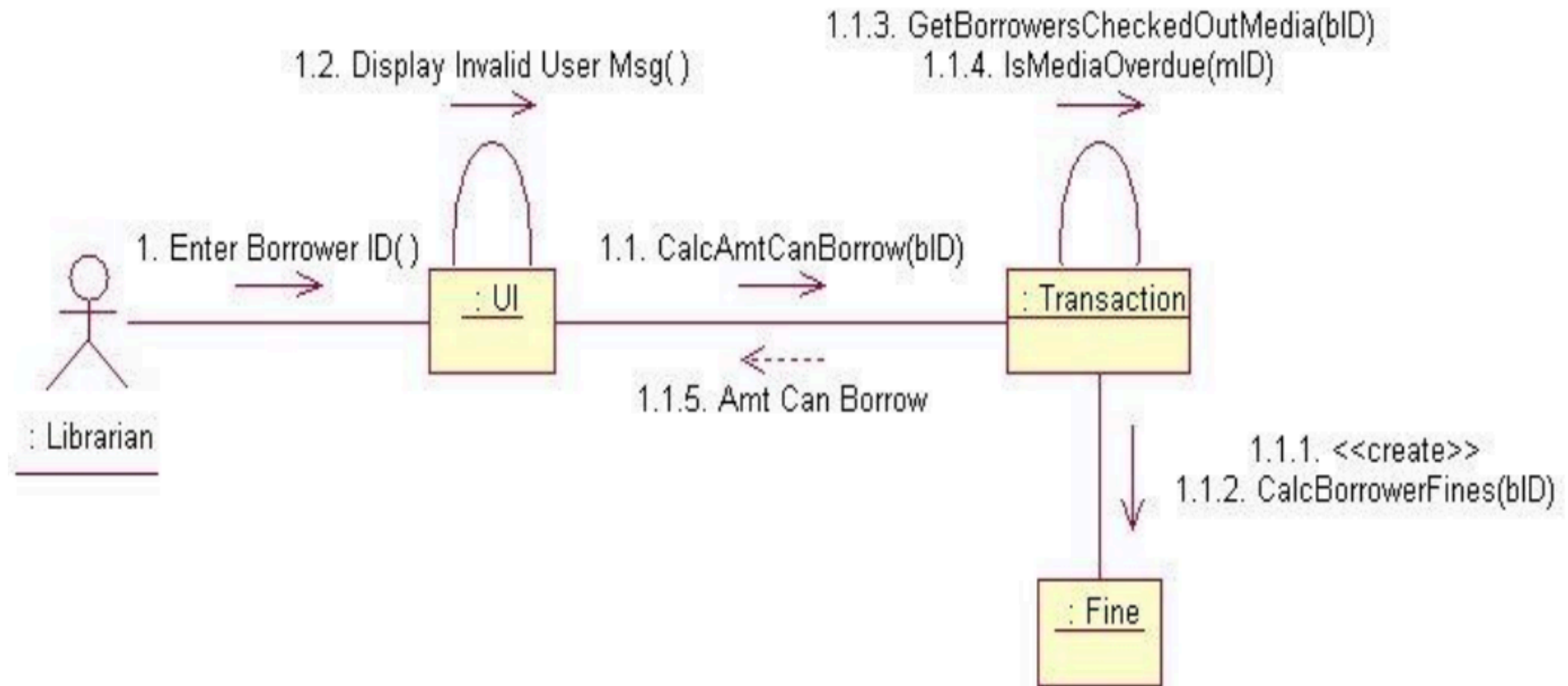
- Because there is no implicit timeline, every message is prefixed with a sequence number.
- The numbers indicate the order of the messages. Message 1 happens before 2, which happens before 3.
- We use decimal numbering (e.g., 1.1, 1.2, 1.2.1) to show nested calls—messages that are sent as part of handling a parent message.

MESSAGES

The Library Example

- 1: Enter Borrower ID is the first message.
- 1.1: CalcAmtCanBorrow(bID) is the first thing that happens inside the context of message 1.
- 1.1.1: <<create>> and 1.1.2: CalcBorrowerFines(bID) are nested within message 1.1.
- Meanwhile, 1.2: GetBorrowersCheckedOutMedia(bID) might be happening in parallel or after 1.1.
- 1.2.1: IsMediaOverdue(mID) is nested within message 1.2.

MESSAGES



ITERATIVE MESSAGES

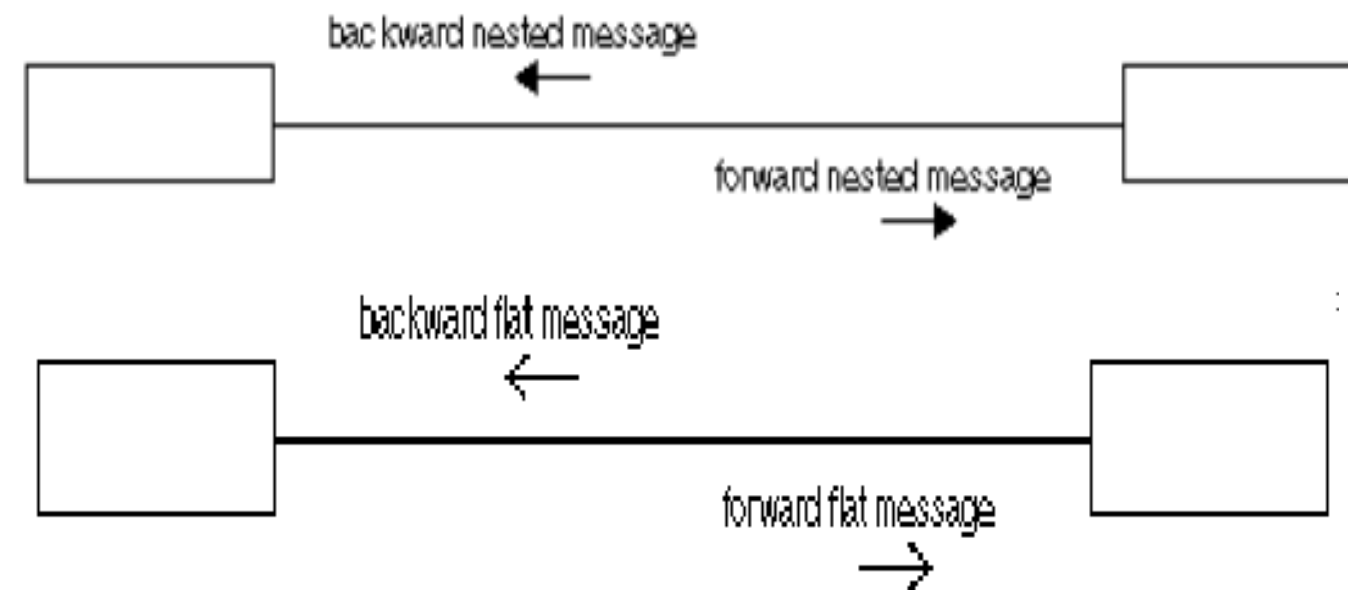
- Similar to sequence diagrams, asterisk * to indicate conditional messages.
- **Notation:** You can prefix the sequence number with a *.
- **Example:** 1.1*: prepare() means 'message 1.1 is repeated multiple times.'
- You can also be more specific: 1.1..5: process() would mean 'repeat message 1.1 five times.'

CONDITIONAL MESSAGES

- Similar to sequence diagrams, we use guards in square brackets [] to indicate conditional messages.
- **Notation:** The guard is placed after the sequence number and before the message.
- **Example:** 1.1.3 [check == true]: new means 'send this message only if the condition is true.' This is the collaboration diagram equivalent of the alt fragment.

NESTED VS. FLAT MESSAGE

- Nested Message (Filled Solid Arrowhead): This represents a nested procedure call. The nested sequence must complete before the outer level continues. This is the most common type and implies synchronous communication.
- Flat Message (Stick Arrowhead): This represents an asynchronous signal. The sender does not wait for a response; it fires the message and continues immediately.



OTHER MESSAGE NOTATIONS

Since collaboration diagrams don't have explicit lifelines, we have special notations for an object's lifecycle.

- Creation: We use a <<create>> stereotype on the message that creates the new object. The new object will appear on the diagram where it is created.
- Deletion: We can use a <<destroy>> message, though it's less common.
- State Change: This is a powerful feature. If an object changes its state significantly during the interaction, you can show this by:
 - Drawing the object a second time on the diagram.
 - Labeling each instance with its state (e.g., Contract [Pending], Contract [Accepted]).
 - Connecting them with a <<become>> message to show the transformation.

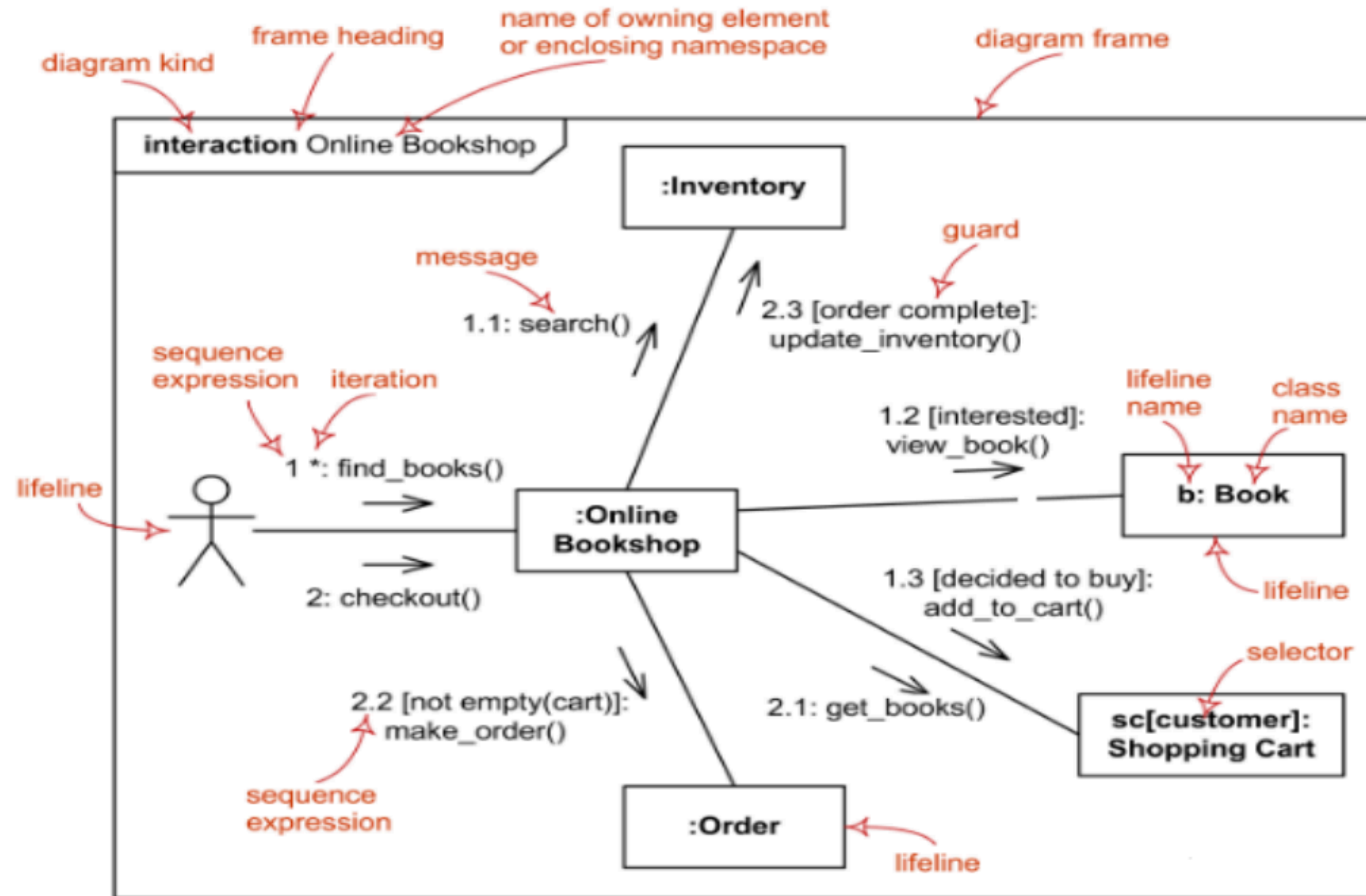
OTHER MESSAGE NOTATIONS



STEPS TO CREATING A COLLABORATION DIAGRAM

- Determine the Scope: Which use case or scenario are you modeling?
- Place the Objects: Identify all participating objects and place them on the diagram. Put the most important ones toward the center.
- Set Initial States: If an object's initial state is important, note it (e.g., Order [Pending]).
- Create Links: Draw lines between objects that communicate. This reflects the associations from your class diagram.
- Add Messages: For each interaction, draw an arrow along the link and label it with the message.
- Add Sequence Numbers: This is the final, critical step. Number the messages to show the correct order and nesting of the interaction.

ONLINE BOOK PURCHASE EXAMPLE





SEQUENCE VS. COLLABORATION DIAGRAMS

- In reality, sequence diagrams and collaboration diagrams show the same information, but just present it differently
- Not used as often as sequence diagrams but are closely related
- Sequence Diagrams are arranged according to Time.
- Collaboration Diagrams represent the structural organization of object.
- [Both sequence and collaboration diagrams are called interaction diagrams]

SEQUENCE VS. COLLABORATION DIAGRAMS

In sequence diagrams, each message icon represents a single message.	In collaboration diagrams, a message icon can represent one or more messages.
Sequence diagram illustrates interaction in a kind of fence format, in which each new object is added to the right. It generally shows the sequence of events that occur.	Collaboration diagram illustrates object interactions in a graph or network format, in which objects can be placed anywhere on the diagram. It demonstrates how objects are statically connected.
One message icon represents a single message.	One message icon on a link can represent multiple messages e.g., 1.1.1 and 1.1.2 on the same arrow.
The difference between sequence diagrams and collaboration diagrams is that collaboration diagrams emphasize more the structure than the sequence of interactions	

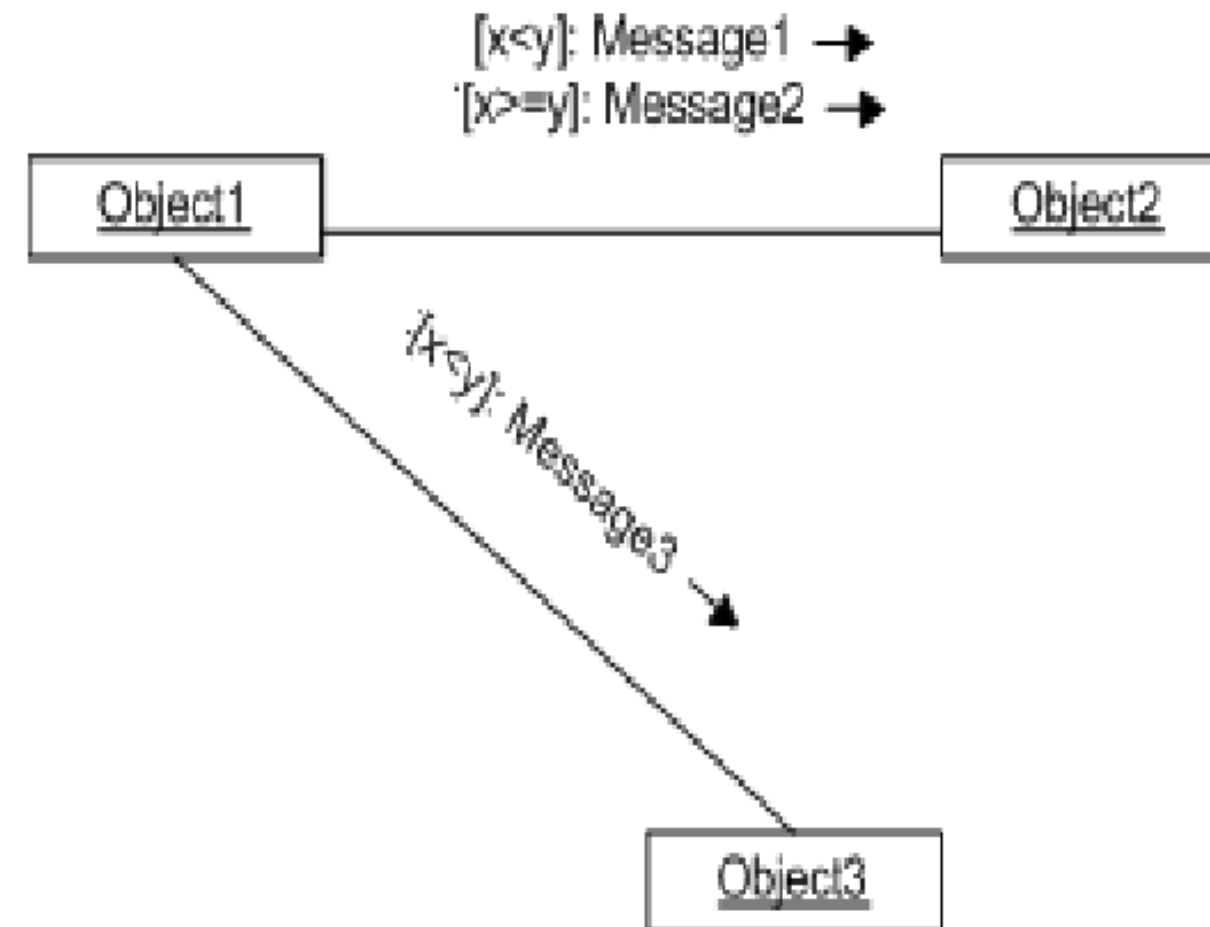


WHY COLLABORATION DIAGRAMS

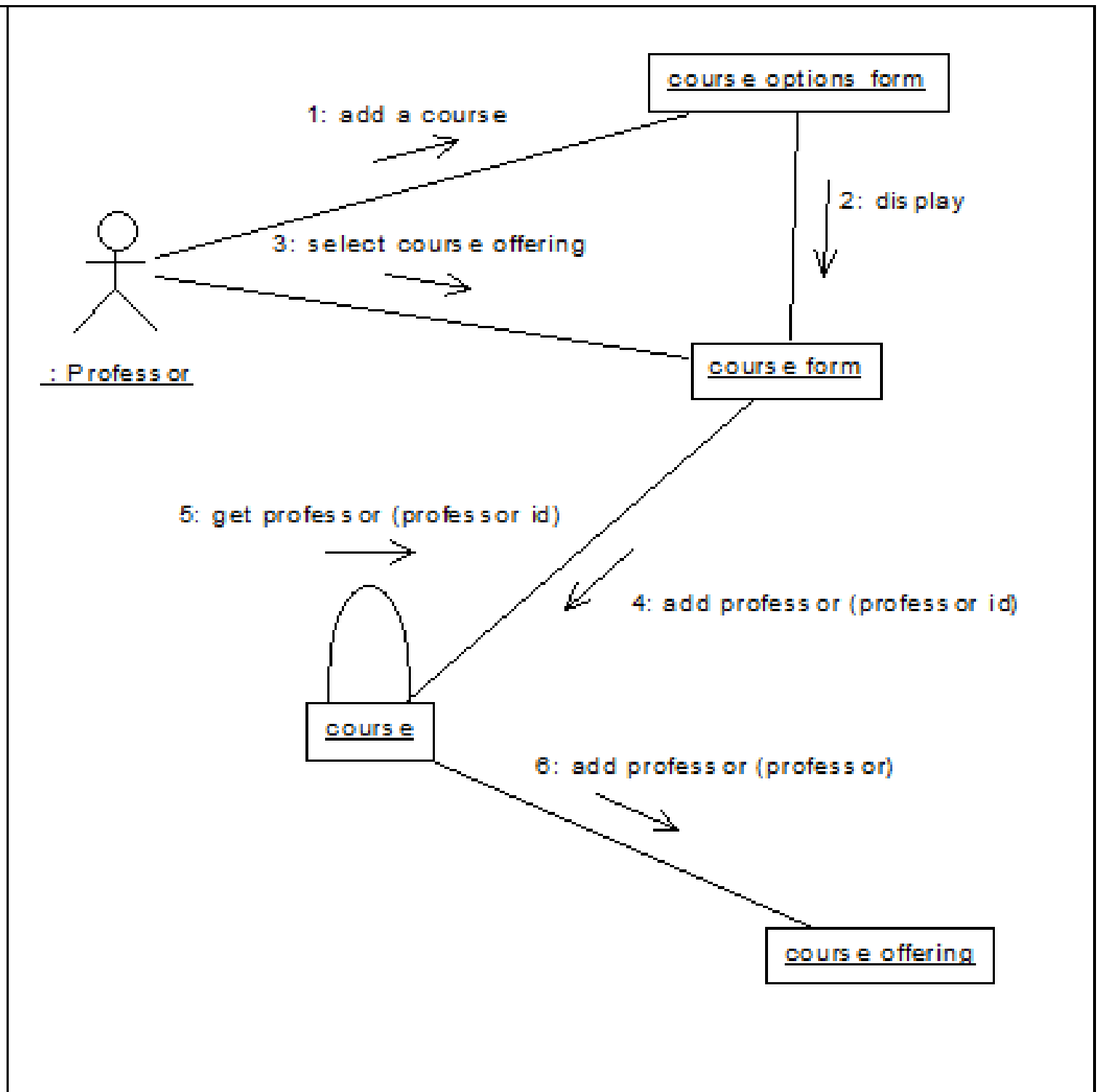
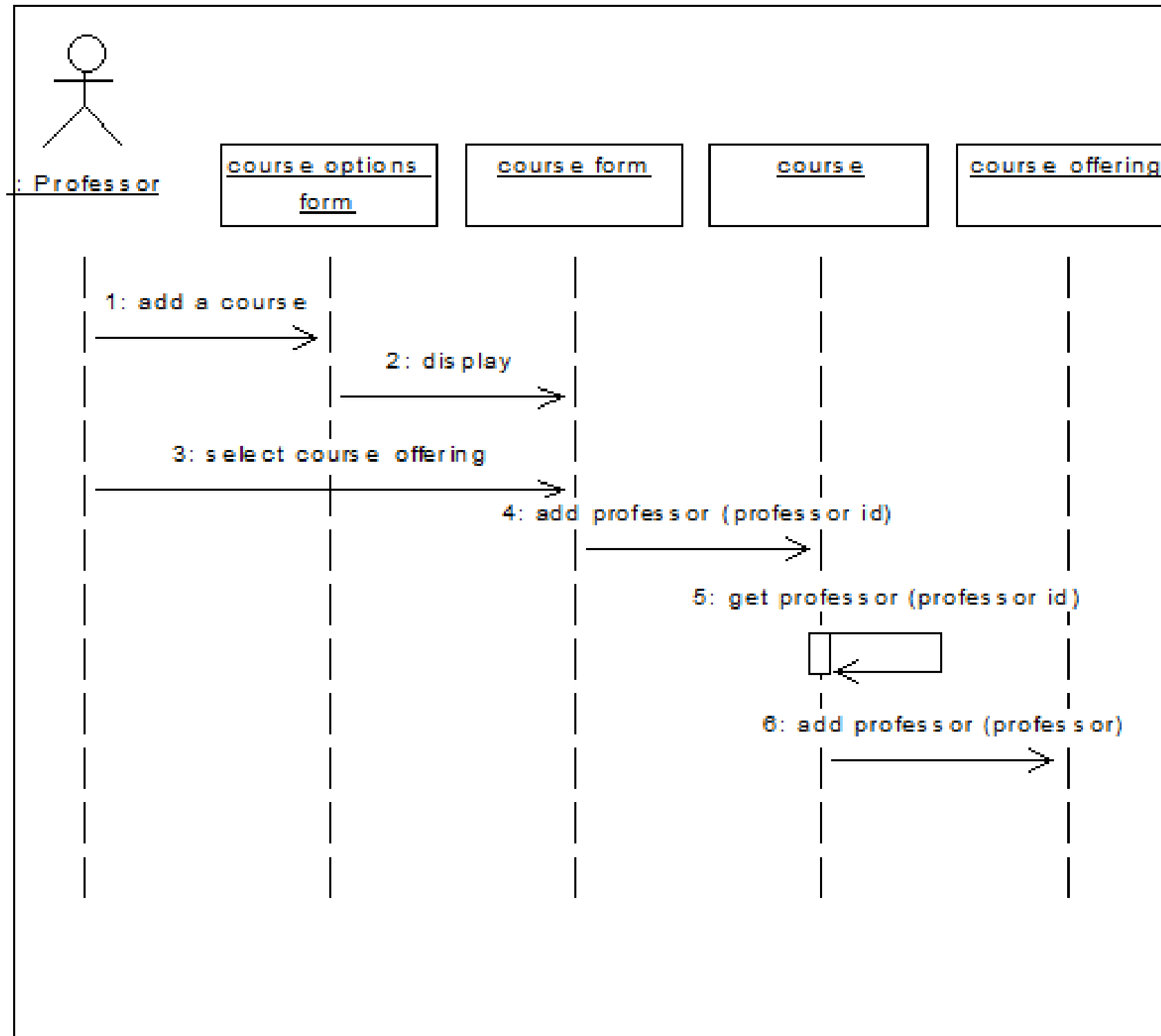
- When Structure is Paramount: If you need to show how objects are statically connected and how those connections are used, a collaboration diagram is superior. A sequence diagram hides this structural information.
- For Procedural Design: When you want to trace all the messages that affect a particular object, the network view is incredibly effective. You can focus on one object and see everything that talks to it.
- When the Sequence is Simple: If the order of messages is straightforward, the collaboration diagram gives you both the sequence and the structure in one concise view

IF-ELSE

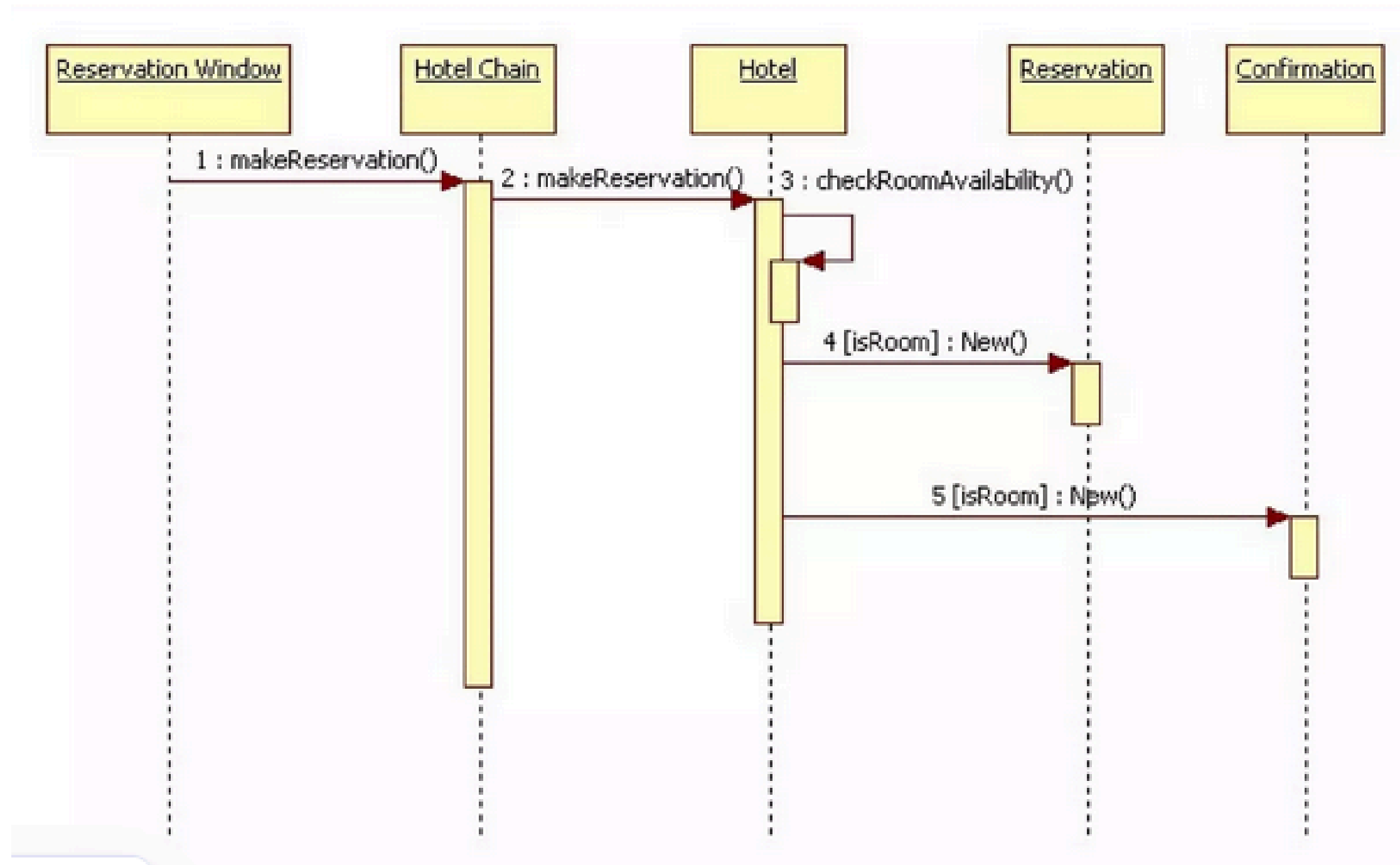
```
if (x<y)
{
    object2.message1();
    object3.message3();
}
else
{
    object2.message2();
}
```



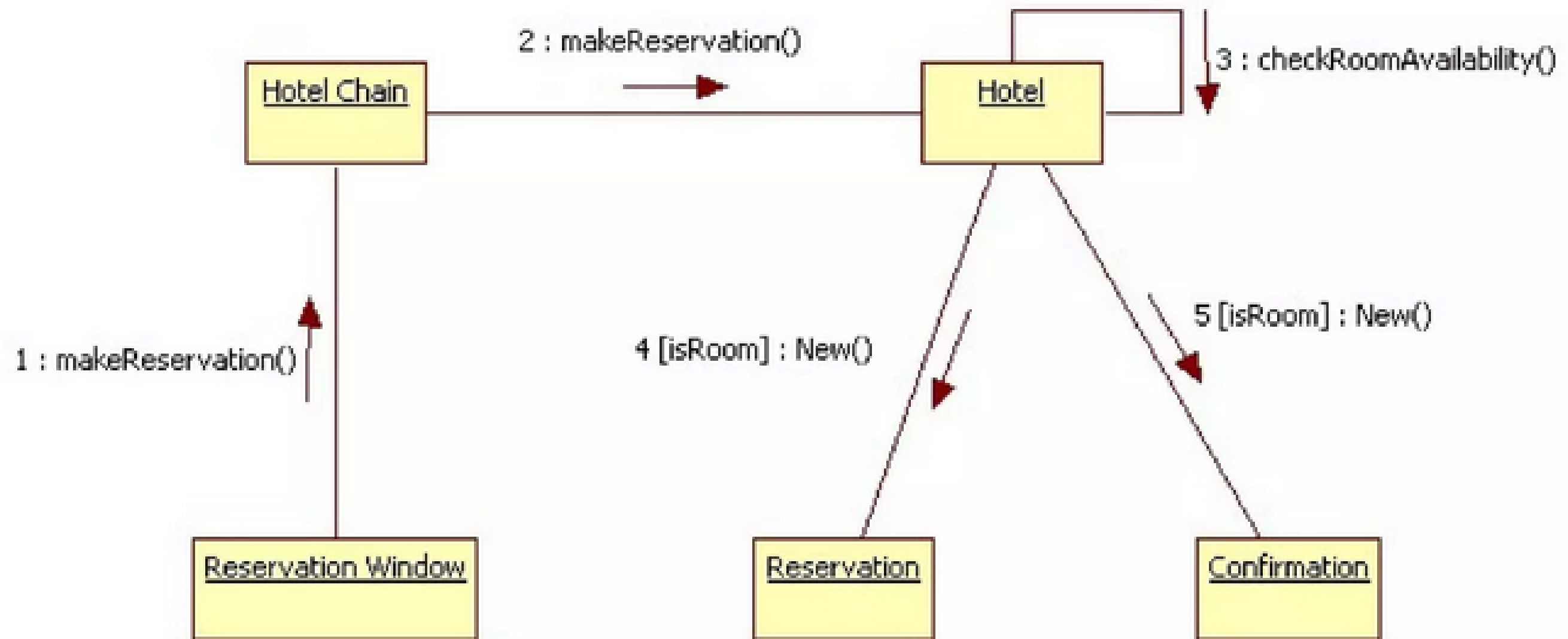
EXAMPLE



EXAMPLE



EXAMPLE

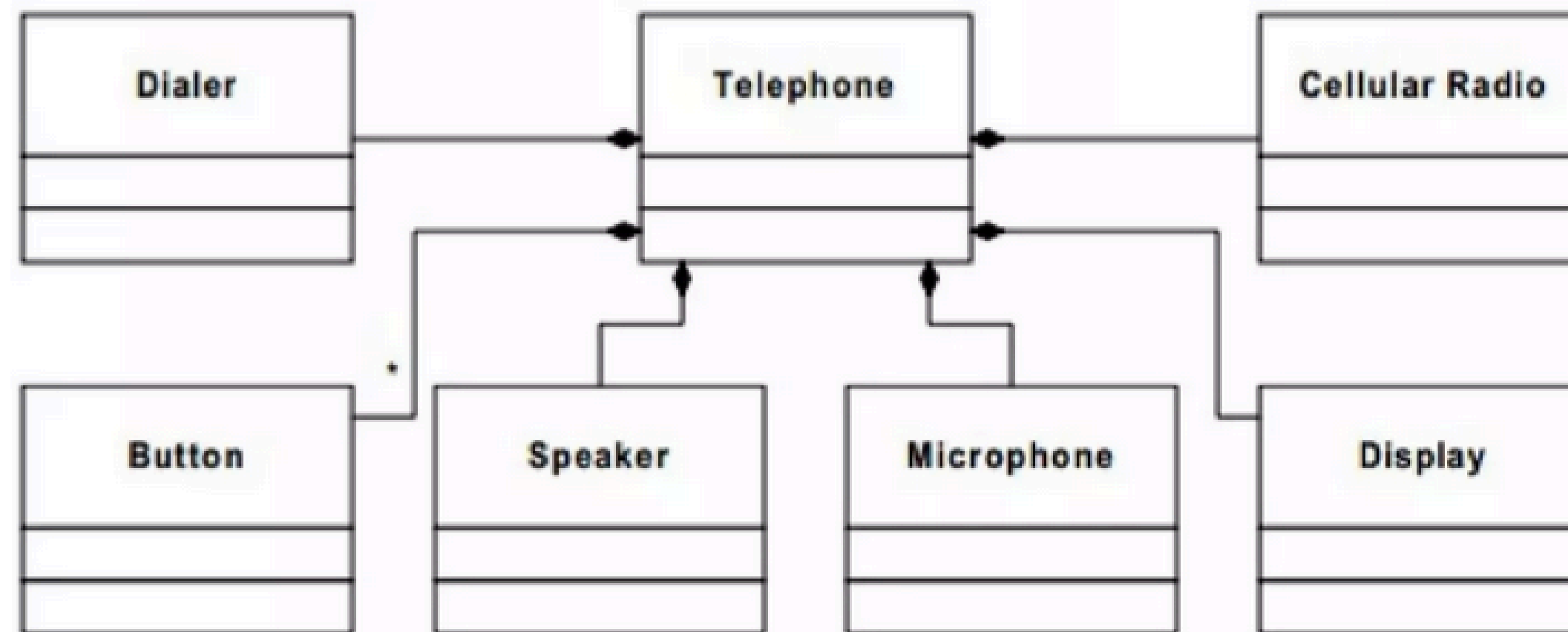


CELLULAR PHONE CALL CLASS DIAGRAM

Example: A Cellular Phone.

Consider the software that controls a very simple cellular telephone. Such a phone has buttons for dialing digits, and a “send” button for initiating a call. It has “dialer” hardware and software that gathers the digits to be dialed and emits the appropriate tones. It has a cellular radio that deals with the connection to the cellular network. It has a microphone, a speaker, and a display.

From this simple spec, we might be tempted to create a static model as shown in Figure 1.



CELLULAR PHONE CALL EXAMPLE ...

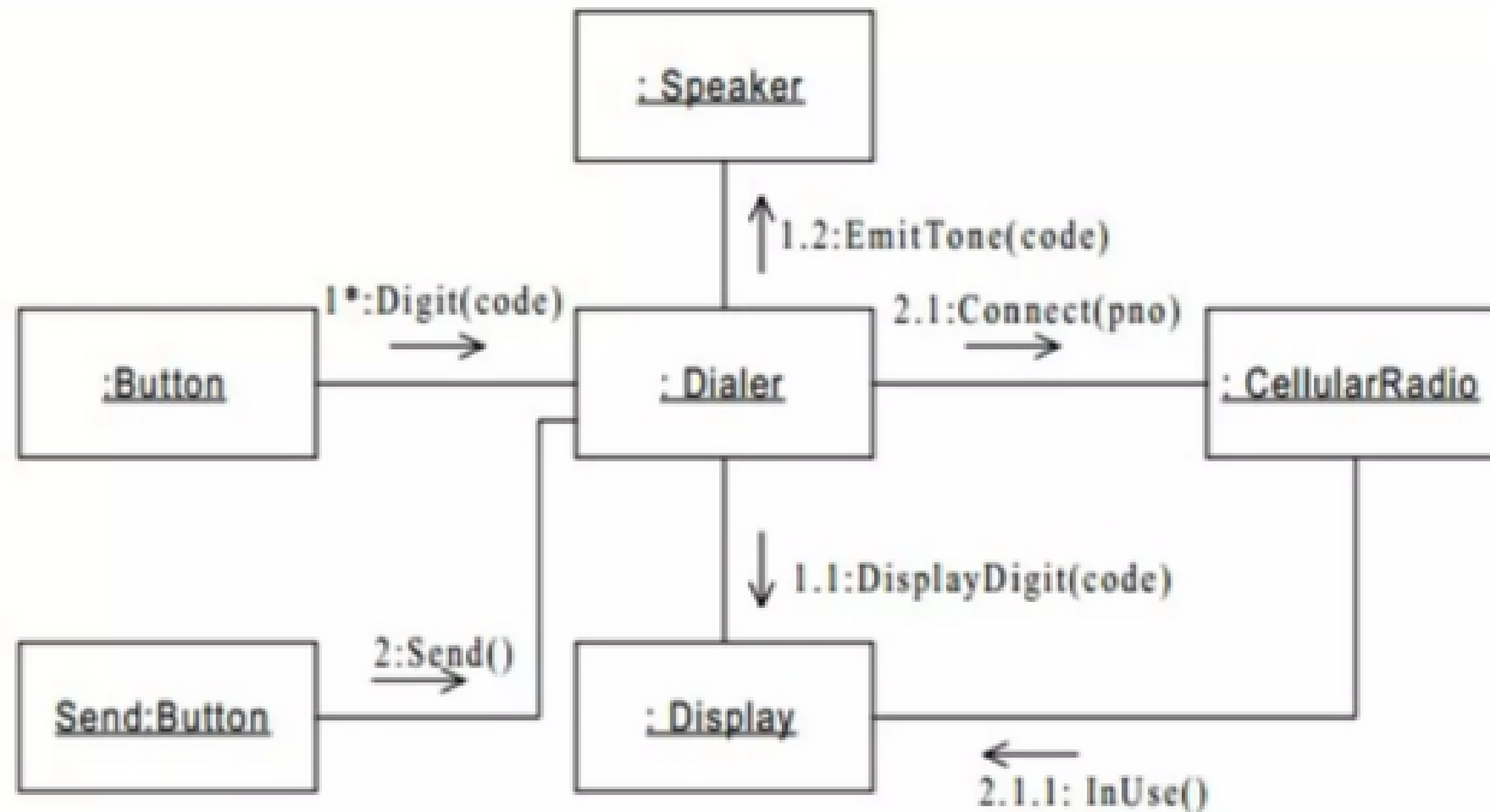
Specifying Dynamics.

How does the cellular phone work? To keep things simple, let's just look at how a customer might make a phone call. The use case for this interaction looks like this:

Use case: Make Phone Call

1. User presses the digit buttons to enter the phone number.
2. For each digit, the display is updated to add the digit to the phone number.
3. For each digit, the dialer generates the corresponding tone and emits it from the speaker.
4. User presses "Send"
5. The "in use" indicator is illuminated on the display
6. The cellular radio establishes a connection to the network.
7. The accumulated digits are sent to the network.
8. The connection is made to the called party.

CELLULAR PHONE CALL EXAMPLE ...





THANK YOU